Universidade de São Paulo

ACH2036 – Métodos Quantitativos para Análise Multivariada

Atividade 2

Regressão Linear Múltipla

Valor do aluguel residencial em cinco cidades brasileiras

Nicholas Kim
Número USP 16990054

Luiz Eduardo da Silva
Número USP 17018992

Paulo Henrique Froelich Gomes
Número USP 15483136

São Paulo, 2026

Sumário

1 Problema de regressão a ser tratado **1**

2 Descrição da base **1**

2.1 Dicionário de variáveis 1

2.2 Leitura dos dados 2

2.3 Registros duplicados e valores implausíveis 3

2.4 Variáveis excluídas, categóricas e variável derivada 4

2.5 Transformações de escala 5

2.6 Atendimento aos critérios de escolha da base 7

3 Caracterização das principais variáveis após os tratamentos **7**

4 Seleção de variáveis **12**

5 Resultados e discussões **14**

5.1 Coeficientes do modelo selecionado 14

5.2 Coeficiente de determinação e teste F 18

5.3 Gráficos dos resíduos 18

5.4 Testes estatísticos sobre os resíduos 22

5.5 Multicolinearidade, observações influentes e erros padrão robustos 23

5.6 Comparação com a escala original e desempenho fora da amostra 26

5.7 Síntese: qualidades, limitações e premissas LINE 28

6 Conclusões **29**

A Funções auxiliares e execução dos blocos **31**

1 Problema de regressão a ser tratado

A base utilizada é a *Brazilian Houses to Rent*, publicada no Kaggle [1] e redistribuída no repositório OpenML com o nome `Brazilian_houses` (ID 42688, versão 4) [2]. A versão do OpenML corresponde ao arquivo `houses_to_rent_v2.csv` da base original e contém 10.692 anúncios de imóveis residenciais para locação em São Paulo, Rio de Janeiro, Belo Horizonte, Porto Alegre e Campinas, coletados por um *web crawler* em sites de anúncios. Cada registro descreve o imóvel (cidade, área, quartos, banheiros, vagas de garagem, andar, aceitação de animais e mobília) e seus custos mensais: aluguel, condomínio, IPTU, seguro contra incêndio e o total desses valores. A única alteração do OpenML em relação ao arquivo original, segundo sua documentação, foi a substituição do andar "-" por 0; de acordo com o autor da base, esse código identifica casas.

O objetivo é avaliar a associação entre o valor mensal do aluguel (em R$) e as características do imóvel, quantificando, por exemplo, quanto o aluguel varia entre as cidades, com a área, com o número de banheiros ou com a presença de mobília, mantidas constantes as demais características. Secundariamente, verifica-se se o modelo permite prever o aluguel de um novo anúncio com acurácia razoável.

A escolha foi motivada pela relevância do tema, pois o custo de moradia é uma preocupação direta de estudantes e trabalhadores nas grandes cidades, e pelas características da base, que exercitam as etapas de pré-processamento descritas no enunciado: resposta e preditoras fortemente assimétricas, variáveis categóricas, contagens, valores nulos legítimos, registros duplicados, valores implausíveis e duas variáveis que, se mantidas, transfeririam a própria resposta para o modelo.

Todo o processamento foi feito em Python, com as bibliotecas pandas, SciPy, statsmodels [9] e Matplotlib. Conforme o enunciado, os scripts são apresentados em blocos ao longo do texto, cada um seguido de seus resultados parciais e da discussão correspondente. Os blocos são executados em sequência e compartilham as variáveis, como as células de um notebook; o script que os executa e as funções auxiliares de visualização estão no Apêndice A. As tabelas foram exportadas diretamente dos resultados, e os valores foram arredondados somente para apresentação.

2 Descrição da base

2.1 Dicionário de variáveis

A Tabela 1 apresenta as variáveis da base. Os nomes originais foram substituídos por abreviações em português, utilizadas nos gráficos, nas tabelas e no código. A base não possui campos identificadores, como código do anúncio, endereço ou nome do anunciante; portanto, nenhuma coluna precisou ser suprimida por esse motivo.

Tabela 1: Dicionário das variáveis.

Abreviação	Nome original	Descrição	Tipo	Uso
aluguel	`rent_amount_(BRL)`	Valor mensal do aluguel (R$)	Contínua	Resposta
cidade	`city`	Cidade do imóvel: São Paulo, Rio de Janeiro, Belo Horizonte, Porto Alegre ou Campinas	Categórica não ordinal	Preditora
area	`area`	Área do imóvel (m^2), registrada em valores inteiros	Contínua	Preditora
quartos	`rooms`	Número de quartos	Inteira	Preditora
banheiros	`bathroom`	Número de banheiros	Inteira	Preditora
vagas	`parking_spaces`	Número de vagas de garagem	Inteira	Preditora
andar	`floor`	Andar do imóvel; "-" (0 no OpenML) para casas	Inteira	Preditora
animal	`animal`	Aceitação de animais de estimação: Aceita ou Não aceita	Categórica não ordinal (binária)	Preditora
mobilia	`furniture`	Mobília: Mobiliado ou Não mobiliado	Categórica não ordinal (binária)	Preditora
condominio	`hoa_(BRL)`	Valor mensal do condomínio (R$)	Contínua	Preditora
iptu	`property_tax_(BRL)`	Valor mensal do IPTU (R$)	Contínua	Preditora
seguro	`fire_insurance_(BRL)`	Valor mensal do seguro contra incêndio (R$)	Contínua	Excluída
total	`total_(BRL)`	Soma de aluguel, condomínio, IPTU e seguro (R$)	Contínua	Excluída
tipo	derivada de `floor`	Casa (andar "-") ou Apartamento	Categórica não ordinal (binária)	Preditora

2.2 Leitura dos dados

O Bloco 1 importa as bibliotecas e lê o arquivo ARFF obtido no OpenML. As variáveis nominais do formato ARFF são lidas como sequências de bytes e, por isso, são convertidas para texto.

Bloco 1: Importações e leitura da base.

```
import numpy as np
import pandas as pd
from scipy import stats
from scipy.io import arff
import statsmodels.formula.api as smf
from statsmodels.stats.anova import anova_lm

import graficos as gr

# Brazilian_houses (OpenML, ID 42688) = houses_to_rent_v2.csv (Kaggle)
```

```
dados, meta = arff.loadarff("dados/Brazilian_houses.arff")
bruto = pd.DataFrame(dados)
for coluna in bruto.select_dtypes(include="object"):
    bruto[coluna] = bruto[coluna].str.decode("utf-8")

print(bruto.shape)
print(bruto.dtypes)
print(bruto.head())
```

A base contém 10.692 registros e 13 colunas, sem valores faltantes. As colunas numéricas são lidas como reais; `floor` é lida como texto, pois o arquivo a declara como atributo nominal, e é convertida para inteiro no próximo bloco.

2.3 Registros duplicados e valores implausíveis

O Bloco 2 renomeia as colunas e trata dois problemas de qualidade dos registros.

Bloco 2: Renomeação, duplicatas e valores implausíveis.

```
NOMES = {
    "city": "cidade", "area": "area", "rooms": "quartos", "bathroom": "banheiros",
    "parking_spaces": "vagas", "floor": "andar", "animal": "animal", "furniture": "
    mobilia",
    "hoa_(BRL)": "condominio", "rent_amount_(BRL)": "aluguel", "property_tax_(BRL)": "
    iptu",
    "fire_insurance_(BRL)": "seguro", "total_(BRL)": "total",
}
df = bruto.rename(columns=NOMES)
df["andar"] = df["andar"].astype(int)
n_original = len(df)

# Registros idênticos em todas as 13 colunas (anúncio coletado mais de uma vez)
duplicado = df.duplicated()
df = df[~duplicado]

# Valores fisicamente ou economicamente implausíveis (erros de digitação ou de coleta)
implausivel = (
    (df["area"] > 10_000) | (df["andar"] > 100)
    | (df["condominio"] > 20_000) | (df["iptu"] > 10_000)
)
excluidos = df.loc[implausivel, ["cidade", "area", "quartos", "andar", "condominio", "
    aluguel", "iptu"]]
print(excluidos)
df = df[~implausivel].reset_index(drop=True)

print(f"Original: {n_original} | duplicados: {duplicado.sum()} | "
      f"implausíveis: {implausivel.sum()} | final: {len(df)}")
```

Foram encontrados **358 registros duplicados**, idênticos nas 13 colunas, inclusive nos quatro valores monetários. A explicação mais plausível é a coleta repetida do mesmo anúncio pelo *crawler*. Mantê-los daria peso dobrado a esses imóveis e introduziria uma dependência artificial entre observações; por isso, foi mantida apenas uma ocorrência de cada. Não é possível excluir completamente a hipótese de unidades distintas com características idênticas no mesmo edifício, mas a coincidência exata de todos os campos torna a repetição a explicação mais provável.

Em seguida, foram excluídos **14 registros implausíveis** (Tabela 2), definidos por quatro critérios objetivos: área superior a 10.000 m^2, andar superior a 100, condomínio superior a R$ 20.000 mensais ou IPTU superior a R$ 10.000 mensais. Os limites foram posicionados em descontinuidades claras das distribuições: após a exclusão, os maiores valores restantes são 2.000 m^2, andar 51, condomínio de R$ 15.000 e IPTU de R$ 9.500. Os registros excluídos combinam valores incompatíveis entre si, como um apartamento de 42 m^2 com IPTU de R$ 313.700 por mês ou um condomínio de R$ 1.117.000 para um aluguel de R$ 2.790, o que indica erro de digitação ou de coleta.

Tabela 2: Registros excluídos por valores implausíveis (valores em R$ por mês).

Cidade	Área (m^2)	Quartos	Andar	Condomínio	Aluguel	IPTU	Critério
Belo Horizonte	80	3	301	750	2.600	164	andar
Belo Horizonte	155	1	4	1.117.000	2.790	64	condomínio
Belo Horizonte	46.335	4	11	960	8.500	646	área
Belo Horizonte	260	4	10	3.200	11.900	12.500	IPTU
Campinas	12.732	3	3	700	1.600	96	área
Porto Alegre	42	1	10	32.000	700	40	condomínio
Rio de Janeiro	95	2	11	1.024	3.000	28.120	IPTU
Rio de Janeiro	35	1	1	81.150	4.500	9.900	condomínio
São Paulo	300	4	6	3.500	1.990	10.830	IPTU
São Paulo	42	1	6	690	2.500	313.700	IPTU
São Paulo	24.606	5	12	2.254	8.100	7.859	área
São Paulo	700	4	0	0	10.000	21.880	IPTU
São Paulo	340	5	7	220.000	12.000	1.000	condomínio
São Paulo	285	4	6	200.000	20.000	1.834	condomínio

Valores extremos, mas plausíveis, foram mantidos, como aluguéis de até R$ 45.000 e áreas de até 2.000 m^2. Assim como na Atividade 1, um valor extremo não foi excluído automaticamente; a exclusão ficou restrita a registros incompatíveis com a realidade. A base analisada passou a ter **10.320 registros** (96,5% do total original).

2.4 Variáveis excluídas, categóricas e variável derivada

O Bloco 3 remove as variáveis que contêm a resposta, recodifica as categóricas e cria a variável *tipo*.

Bloco 3: Exclusão de variáveis com vazamento e tratamento das categóricas.

```
# Seguro e total são calculados a partir do próprio aluguel: vazamento da resposta
print(df[["aluguel", "seguro", "total"]].corr().round(3))
print((df["seguro"] / df["aluguel"]).describe().round(4))
df = df.drop(columns=["seguro", "total"])

# Categorias com nomes em português e categorias de referência na primeira posição
df["cidade"] = pd.Categorical(
    df["cidade"].replace({"Sao Paulo": "São Paulo"}),
    categories=["São Paulo", "Rio de Janeiro", "Belo Horizonte", "Porto Alegre", "
    Campinas"])
df["animal"] = pd.Categorical(df["animal"].map({"acept": "Aceita", "not acept": "Não
    aceita"}),
```

```
                                categories=["Aceita", "Não aceita"])
df["mobilia"] = pd.Categorical(df["mobilia"].map({"not furnished": "Não mobiliado", "
    furnished": "Mobiliado"}),
                                categories=["Não mobiliado", "Mobiliado"])

# Andar "-" (codificado como 0 no OpenML) indica casa; nos apartamentos, andar >= 1
df["tipo"] = pd.Categorical(np.where(df["andar"] == 0, "Casa", "Apartamento"),
                            categories=["Apartamento", "Casa"])

for coluna in ["cidade", "tipo", "animal", "mobilia"]:
    print(df[coluna].value_counts(), "\n")
print(pd.crosstab(df["tipo"], df["condominio"] == 0, margins=True))
print(pd.crosstab(df["tipo"], df["iptu"] == 0, margins=True))
```

Variáveis excluídas. O seguro contra incêndio tem correlação de 0,987 com o aluguel, e a razão seguro/aluguel é praticamente constante: mediana de 1,31%, com 50% dos registros entre 1,28% e 1,50%. Isso indica que o seguro é calculado a partir do próprio aluguel. O total, por definição, contém o aluguel (correlação de 0,961). Usar essas variáveis como preditoras produziria um R^2 artificialmente elevado, sem conteúdo explicativo, pois elas transferem a própria resposta para o modelo. Por isso, foram excluídas.

Variáveis categóricas. As categorias foram renomeadas para o português, e a primeira categoria de cada variável foi definida como referência: São Paulo, a cidade com mais registros, Aceita, Não mobiliado e Apartamento. Não há categorias do tipo "Não informado", e todas as categorias têm frequência relativa de pelo menos 8% (Figura 3); portanto, não foi necessário agrupar categorias raras. As cinco cidades correspondem a mercados imobiliários distintos e não foram consideradas equivalentes.

Variável derivada. O andar "-" (0 no OpenML) identifica 2.370 casas. Foi criada a variável binária *tipo* (Casa ou Apartamento), e *andar* foi mantida como numérica, com valor 0 nas casas. Como $andar = 0$ ocorre se, e somente se, o imóvel é uma casa, o coeficiente de *andar* representa o efeito do andar entre os apartamentos, e o coeficiente de *tipo* compara uma casa com um apartamento extrapolado para o andar 0.

Valores nulos. Há 2.288 registros com condomínio igual a zero, dos quais 2.016 são casas (85% das casas), e 1.480 registros com IPTU igual a zero. Esses zeros podem ser legítimos (casas fora de condomínio, imóveis isentos de IPTU) ou indicar valor não informado ou incluído no aluguel; a base não permite distinguir esses casos. Como o enunciado não prevê tratamento de valores faltantes e a proporção é alta, os zeros foram mantidos como valores observados.

2.5 Transformações de escala

O Bloco 4 calcula a assimetria de cada variável numérica e o expoente ótimo λ^* da transformação Box-Cox [5], estimado por máxima verossimilhança. Como a transformação exige valores positivos, as variáveis com zeros foram deslocadas em uma unidade ($X + 1$) antes da estimação.

Bloco 4: Assimetria, Box-Cox e transformações adotadas.

```
NUMERICAS = ["aluguel", "area", "quartos", "banheiros", "vagas", "andar", "condominio",
     "iptu"]

linhas = []
for v in NUMERICAS:
    x = df[v].astype(float)
    desloc = 1.0 if (x == 0).any() else 0.0  # Box-Cox exige valores positivos
    _, lam = stats.boxcox(x + desloc)
    linhas.append(dict(variavel=v, zeros=int((x == 0).sum()), deslocamento=desloc,
                       assimetria=stats.skew(x), lambda_bc=lam,
                       assim_bc=stats.skew(stats.boxcox(x + desloc, lam)),
                       assim_log10=stats.skew(np.log10(x + desloc)) if (x + desloc).
    min() > 0 else np.nan))
tabela_boxcox = pd.DataFrame(linhas).set_index("variavel")
print(tabela_boxcox.round(3))

# Transformações adotadas
LAMBDA_COND = tabela_boxcox.loc["condominio", "lambda_bc"]
LAMBDA_IPTU = tabela_boxcox.loc["iptu", "lambda_bc"]
df["log_aluguel"] = np.log10(df["aluguel"])
df["log_area"] = np.log10(df["area"])
df["bc_cond"] = stats.boxcox(df["condominio"] + 1.0, LAMBDA_COND)
df["bc_iptu"] = stats.boxcox(df["iptu"] + 1.0, LAMBDA_IPTU)

TRANSFORMADAS = ["log_aluguel", "log_area", "quartos", "banheiros", "vagas", "andar",
    "bc_cond", "bc_iptu"]
print(df[TRANSFORMADAS].agg(["mean", "std", "min", "max", "skew"]).T.round(3))
```

Tabela 3: Assimetria antes e depois das transformações. BC: Box-Cox com o λ^* estimado; para vagas, andar, condomínio e IPTU, BC e logaritmo foram calculados sobre $X + 1$.

Variável	Zeros	Assim.	λ^*	Assim. BC	Assim. $\log_{10}$	Adotada	Assim. final
Aluguel	0	1,804	-0,129	0,025	0,218	$\log_{10}$	0,218
Área	0	2,828	-0,155	0,022	0,289	$\log_{10}$	0,289
Quartos	0	0,691	0,413	-0,049	-0,416	nenhuma	0,691
Banheiros	0	1,183	-0,344	0,094	0,300	nenhuma	1,183
Vagas	2.486	1,466	-0,081	0,012	0,100	nenhuma	1,466
Andar	2.370	1,395	0,082	-0,030	-0,129	nenhuma	1,395
Condomínio	2.288	2,637	0,277	-0,358	-1,071	BC($X + 1$)	-0,358
IPTU	1.480	4,570	0,183	-0,084	-0,882	BC($X + 1$)	-0,084

A Tabela 3 e a Figura 1 mostram que aluguel, área, condomínio e IPTU têm assimetria positiva forte, com caudas longas à direita. As decisões foram as seguintes:

- **Aluguel e área:** λ^* próximo de zero ($-0{,}129$ e $-0{,}155$). Foi adotado $\lambda = 0$, isto é, $\log_{10}$, que reduz a assimetria para 0,218 e 0,289. O λ^* exato produziria assimetria ainda menor, mas o logaritmo permite interpretar os coeficientes como variações percentuais e, no caso da área, como elasticidade. Também mantém a preferência pelo $\log_{10}$ da Atividade 1.
- **Condomínio e IPTU:** o $\log_{10}(X + 1)$ produz assimetria negativa ($-1{,}071$ e $-0{,}882$), pois afasta excessivamente os zeros dos demais valores. A transformação Box-Cox com $\lambda^* = 0{,}277$ e $\lambda^* = 0{,}183$ resulta em distribuições muito mais simétricas ($-0{,}358$ e $-0{,}084$)

e foi adotada. Nenhuma transformação monótona elimina a concentração de valores em zero, que permanece como um pico isolado (Figura 2).

- **Quartos, banheiros, vagas e andar:** mantidos na escala original. São contagens com poucos valores distintos, cujos coeficientes são interpretados diretamente (efeito de uma unidade adicional). A regressão não exige normalidade das preditoras: a transformação de uma preditora só se justifica se melhorar a linearidade da relação ou reduzir a alavancagem de valores extremos. A forma linear dessas variáveis é verificada nos gráficos de resíduos da Seção 5.3.

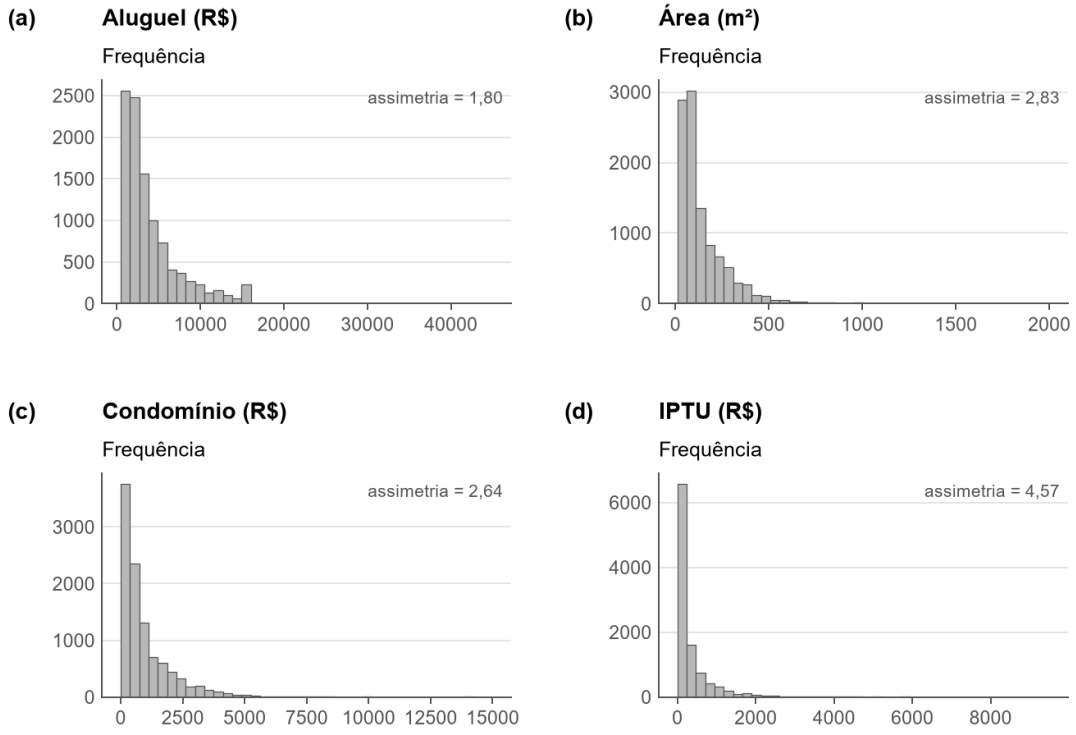


Figura 1: Distribuições das variáveis monetárias e da área na escala original, após a limpeza.

2.6 Atendimento aos critérios de escolha da base

A base atende aos três critérios do enunciado: a resposta (aluguel) é contínua; há 11 preditoras candidatas, sendo dez originais, após a exclusão de seguro e total, e a variável derivada *tipo*; e quatro delas são categóricas (cidade, tipo, animal e mobília). Como o número de candidatas é ligeiramente superior a dez, a seleção de variáveis da Seção 4 é aplicada a todas elas.

3 Caracterização das principais variáveis após os tratamentos

O Bloco 5 gera os gráficos da análise exploratória sobre os dados pré-processados e as estatísticas por categoria.

Bloco 5: Gráficos e tabelas da análise exploratória.

```
ROTULOS = {
    "log_aluguel": "log10(Aluguel)", "log_area": "log10(Área)", "quartos": "Quartos",
    "banheiros": "Banheiros", "vagas": "Vagas", "andar": "Andar",
    "bc_cond": "Box-Cox(Condomínio)", "bc_iptu": "Box-Cox(IPTU)",
}
CATEGORICAS = {"cidade": "Cidade", "tipo": "Tipo de imóvel", "animal": "Animais", "
    mobilia": "Mobília"}

gr.histogramas(df, {"aluguel": "Aluguel (R$)", "area": "Área (m²)", "condominio": "
    Condomínio (R$)",
                    "iptu": "IPTU (R$)"}, "hist_originais", colunas_grade=2)
gr.histogramas(df, ROTULOS, "hist_transformadas")
gr.barras(df, CATEGORICAS, "barras_categoricas")
gr.matriz_dispersao(df, ROTULOS, "matriz_dispersao")
gr.boxplots(df, "log_aluguel", "log10(Aluguel)", CATEGORICAS, "boxplots_categoricas")

print(df[TRANSFORMADAS].corr().round(3))
for coluna in CATEGORICAS:
    print(df.groupby(coluna, observed=True)["aluguel"].agg(["count", "median", "mean"])
    .round(0), "\n")
```

Distribuições. Na escala $\log_{10}$, o aluguel tem distribuição aproximadamente simétrica (Figura 2a), com picos associados a valores arredondados (R$ 1.500, R$ 2.000 etc.) e uma concentração no extremo direito, em $\log_{10}(15.000) \approx 4{,}18$. Há **231 anúncios com aluguel exatamente igual a R$ 15.000** e somente 12 acima desse valor, o que sugere um limite superior na coleta ou no filtro de busca utilizado pelo *crawler*. A documentação da base não menciona esse limite; seu efeito sobre o ajuste é avaliado na Seção 5.3. A área em $\log_{10}$ é aproximadamente simétrica. Banheiros, vagas e andar continuam assimétricos, como esperado para contagens, e condomínio e IPTU transformados apresentam o pico em zero discutido anteriormente.

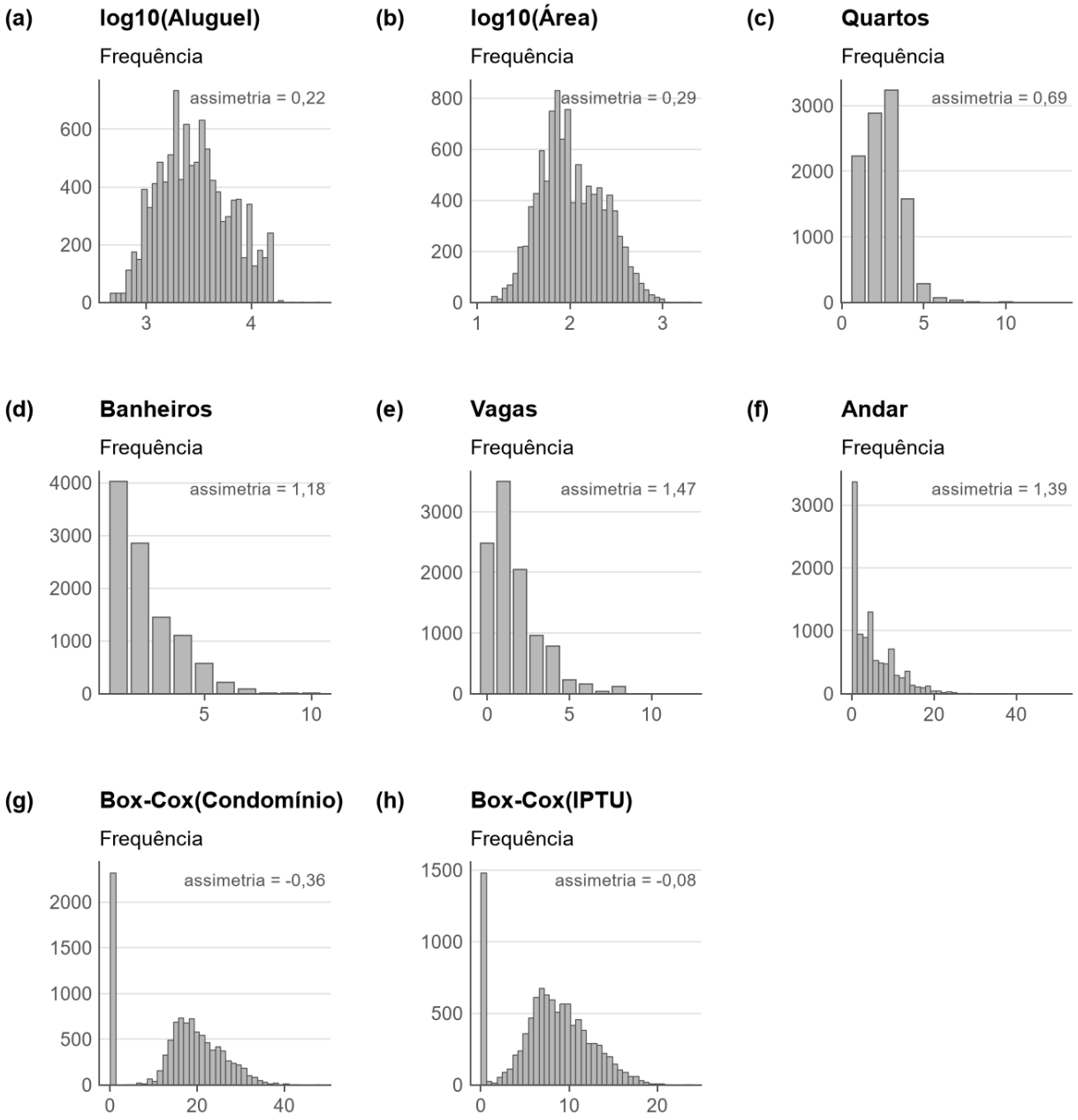


Figura 2: Distribuições das variáveis numéricas na escala utilizada no modelo. Contagens são exibidas em gráficos de barras, com uma barra por valor.

Variáveis categóricas. São Paulo concentra 55,3% dos registros, e a menor categoria é Campinas, com 8,0% (Figura 3). Há 77,0% de apartamentos, 78,1% dos imóveis aceitam animais e 24,3% são mobiliados. As medianas do aluguel (Figura 4) são de R$ 3.500 em São Paulo, R$ 2.400 no Rio de Janeiro, R$ 2.350 em Belo Horizonte, R$ 1.620 em Porto Alegre e R$ 1.500 em Campinas. Casas têm mediana maior que apartamentos (R$ 3.900 e R$ 2.500), e imóveis mobiliados, maior que não mobiliados (R$ 3.600 e R$ 2.460). Essas diferenças são marginais: refletem também diferenças de área e de padrão entre os grupos, e somente o modelo múltiplo permite compará-las com as demais características constantes.

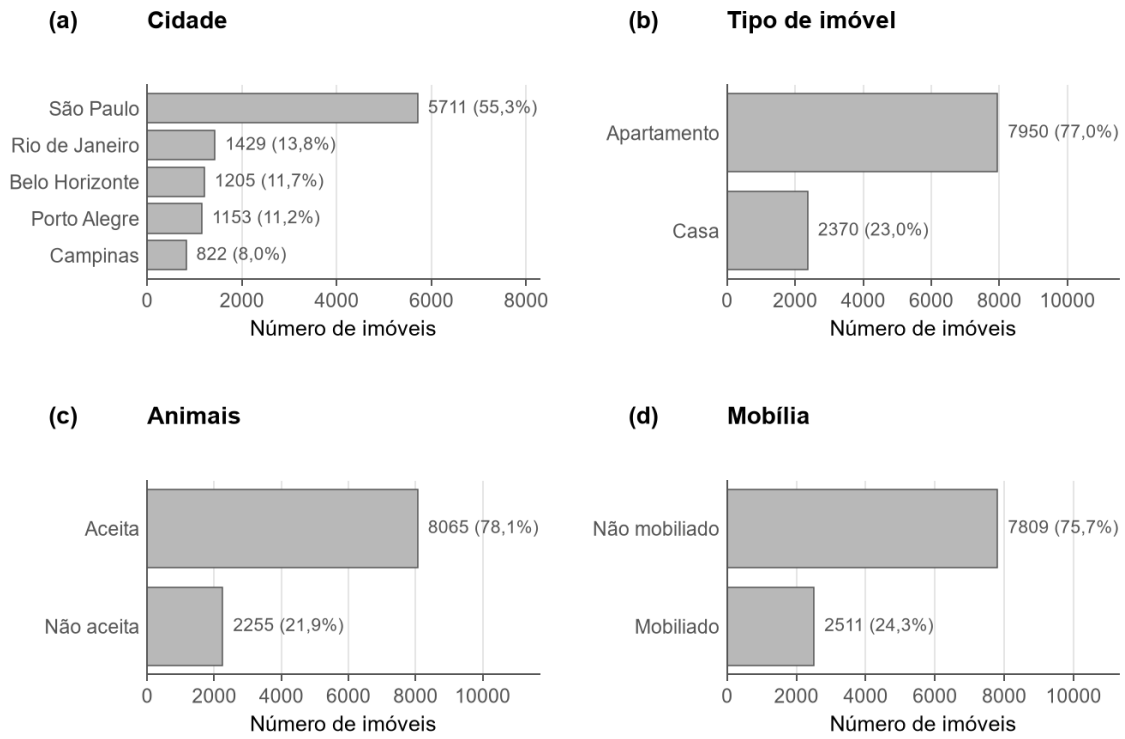


Figura 3: Frequências absolutas e relativas das variáveis categóricas.

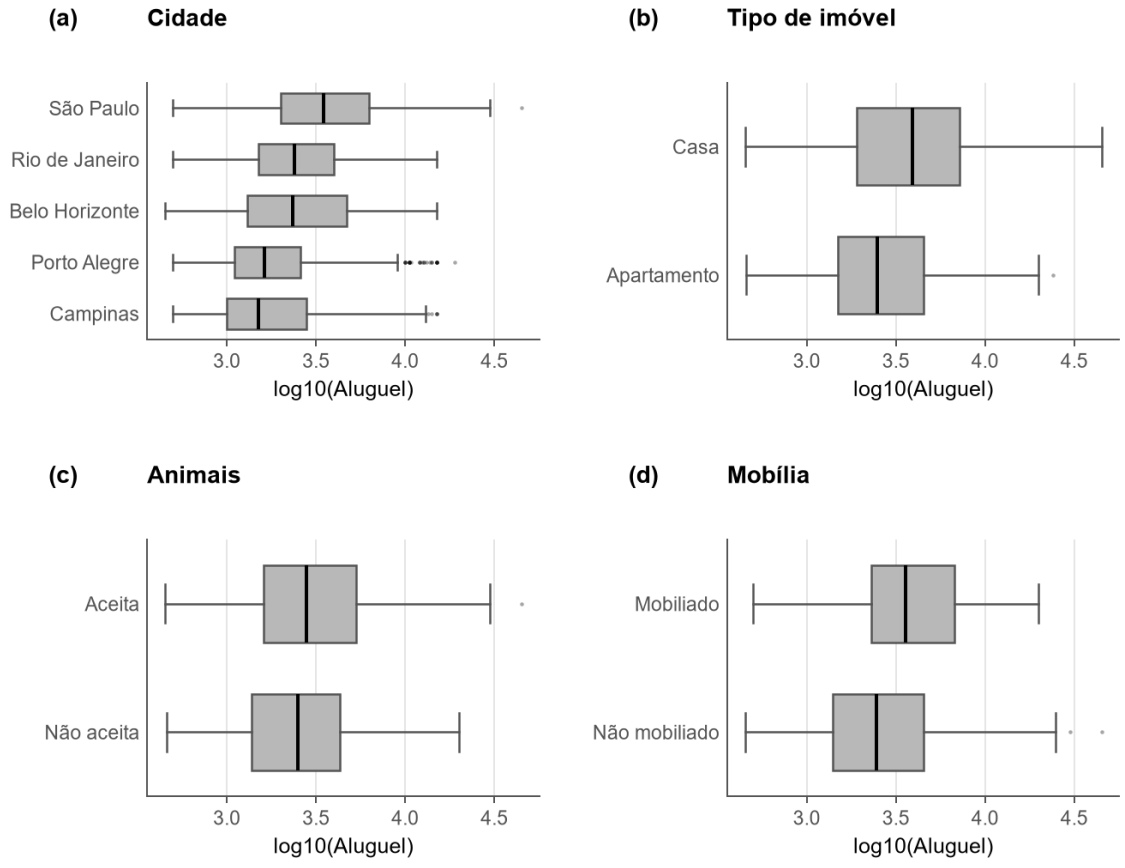


Figura 4: $\log_{10}$(Aluguel) por categoria, com categorias ordenadas pela mediana.

Associações entre as variáveis numéricas. A Figura 5 apresenta uma matriz de dispersão adaptada: abaixo da diagonal estão as dispersões de uma amostra de 3.000 registros, com pequeno deslocamento aleatório nas contagens para evitar sobreposição; na diagonal, os histogramas; e acima, os coeficientes de correlação de Pearson calculados com todos os registros. As maiores correlações com $\log_{10}$(Aluguel) são as de $\log_{10}$(Área) (0,73), banheiros (0,69), IPTU transformado (0,61), vagas (0,60) e quartos (0,59). As relações são aproximadamente lineares e crescentes.

Entre as preditoras, as variáveis de tamanho são fortemente correlacionadas: a área em escala logarítmica tem correlação de 0,81 com quartos e de 0,80 com banheiros, e quartos e banheiros têm correlação de 0,73. Isso antecipa multicolinearidade e sugere que parte dessas variáveis pode ser redundante. O condomínio transformado tem correlação baixa com o aluguel (0,22), mas a dispersão revela dois grupos: os imóveis sem condomínio, em sua maioria casas, formam uma faixa horizontal em zero, enquanto entre os demais a associação é claramente crescente. A correlação de 0,52 entre andar e condomínio decorre do mesmo fenômeno, pois as casas têm zero nas duas variáveis.

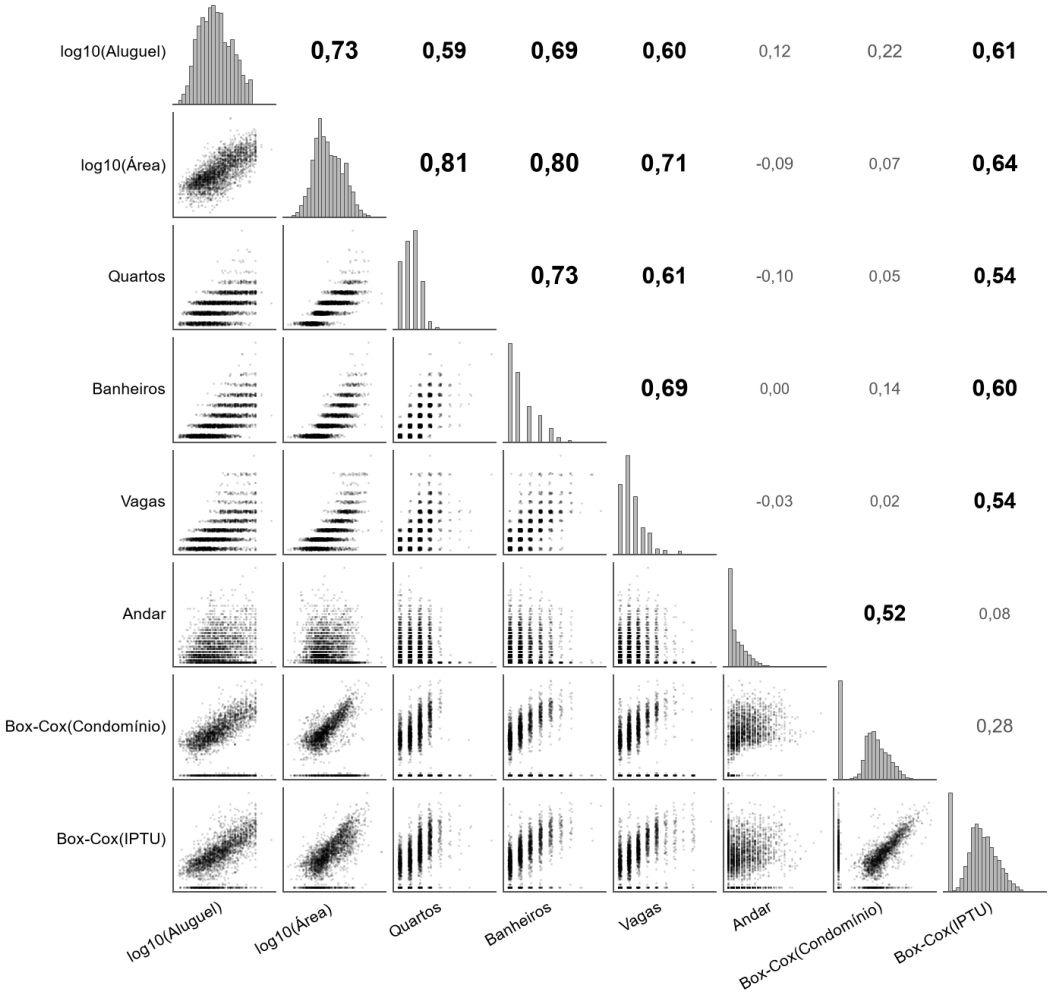


Figura 5: Matriz de dispersão adaptada das variáveis numéricas transformadas: dispersões (abaixo), histogramas (diagonal) e correlações de Pearson (acima; em negrito quando $|r| \geq 0{,}5$).

4 Seleção de variáveis

Foi utilizado o procedimento *stepwise* bidirecional descrito na Lição 10.2 do material STAT 501 [3], com níveis $\alpha_E = \alpha_R = 0{,}15$ para entrada e remoção. O processo parte do modelo somente com intercepto. Em cada passo, a candidata com menor p-valor entra no modelo se $p < \alpha_E$; após cada entrada, os termos já incluídos são reavaliados, e aquele com maior p-valor é removido se $p > \alpha_R$. O processo termina quando nenhuma candidata pode entrar e nenhum termo deve sair.

Como *cidade* tem cinco categorias, ela é representada por quatro variáveis indicadoras, que entram ou saem do modelo em conjunto. Por isso, em vez do teste t de um único coeficiente, utilizou-se o teste F parcial, que compara os modelos com e sem o termo e generaliza o teste t para termos com mais de um grau de liberdade. Com $n = 10.320$, alguns p-valores ficam abaixo do menor número representável em ponto flutuante e são arredondados para zero. Por esse motivo, a ordenação das candidatas foi feita pelo logaritmo do p-valor.

Bloco 6: Seleção *stepwise* por teste F parcial.

```
CANDIDATOS = ["cidade", "tipo", "mobilia", "animal", "log_area", "quartos", "banheiros
    ",
              "vagas", "andar", "bc_cond", "bc_iptu"]
ALFA_E = ALFA_R = 0.15  # STAT501, Lição 10.2


def ajustar(termos: list[str]):
    lado_direito = " + ".join(t for t in CANDIDATOS if t in termos) or "1"
    return smf.ols(f"log_aluguel ~ {lado_direito}", data=df).fit()


def teste_f_parcial(reduzido, completo):
    """Teste F parcial de um termo (uma ou mais colunas): retorna F, p, log(p) e gl.""
    "
    gl = int(round(completo.df_model - reduzido.df_model))
    f = ((reduzido.ssr - completo.ssr) / gl) / (completo.ssr / completo.df_resid)
    return f, stats.f.sf(f, gl, completo.df_resid), stats.f.logsf(f, gl, completo.
    df_resid), gl


selecionados, trajetoria = [], []
for _ in range(50):
    atual = ajustar(selecionados)
    entradas = [(*teste_f_parcial(atual, ajustar(selecionados + [t])), t)
                for t in CANDIDATOS if t not in selecionados]
    if not entradas:
        break
    f, p, logp, gl, termo = min(entradas, key=lambda e: e[2])  # menor p (em escala
    log)
    if p >= ALFA_E:
        break
    selecionados.append(termo)
    trajetoria.append(dict(acao="Entrada", termo=termo, gl=gl, F=f, p=p,
                           r2_aj=ajustar(selecionados).rsquared_adj))
    while len(selecionados) > 1:
        atual = ajustar(selecionados)
        saidas = [(*teste_f_parcial(ajustar([u for u in selecionados if u != s]),
    atual), s)
```

```
                    for s in selecionados]
        f, p, logp, gl, termo = max(saidas, key=lambda e: e[2])  # maior p
        if p <= ALFA_R:
            break
        selecionados.remove(termo)
        trajetoria.append(dict(acao="Saída", termo=termo, gl=gl, F=f, p=p,
                               r2_aj=ajustar(selecionados).rsquared_adj))

trajetoria = pd.DataFrame(trajetoria, index=pd.RangeIndex(1, len(trajetoria) + 1, name
    ="passo"))
print(trajetoria)
nao_selecionados = [t for t in CANDIDATOS if t not in selecionados]
print("Selecionados:", selecionados, "| não selecionados:", nao_selecionados)

completo, final = ajustar(CANDIDATOS), ajustar(selecionados)
print(anova_lm(final, completo))
comparacao = pd.DataFrame({m: dict(termos=len(t), p=int(r.df_model) + 1, R2=r.rsquared,
     R2_aj=r.rsquared_adj,
                                    AIC=r.aic, BIC=r.bic)
                           for m, t, r in [("Completo", CANDIDATOS, completo),
                                           ("Selecionado", selecionados, final)]}).T
print(comparacao)

# Contribuição de cada termo: queda no R2 ao retirá-lo do modelo selecionado
importancia = pd.Series({t: final.rsquared - ajustar([u for u in selecionados if u !=
    t]).rsquared
                         for t in selecionados}).sort_values(ascending=False)
print(importancia.round(4))
```

Tabela 4: Trajetória do *stepwise* ($\alpha_E = \alpha_R = 0{,}15$). Nenhum termo foi removido após entrar.

Passo	Ação	Termo	gl	F parcial	p-valor	R^2_{aj} após
1	Entrada	$\log_{10}$(Área)	1	11.631,34	$< 10^{-300}$	0,5299
2	Entrada	Cidade	4	454,17	$< 10^{-300}$	0,6001
3	Entrada	Mobília	1	1.286,62	$1{,}222 \times 10^{-265}$	0,6444
4	Entrada	Banheiros	1	605,27	$6{,}356 \times 10^{-130}$	0,6641
5	Entrada	BC(IPTU)	1	423,48	$3{,}003 \times 10^{-92}$	0,6773
6	Entrada	Andar	1	301,31	$1{,}504 \times 10^{-66}$	0,6865
7	Entrada	Vagas	1	83,85	$6{,}364 \times 10^{-20}$	0,6890
8	Entrada	BC(Condomínio)	1	19,15	$1{,}217 \times 10^{-5}$	0,6895
9	Entrada	Tipo	1	41,81	$1{,}052 \times 10^{-10}$	0,6907
10	Entrada	Animais	1	8,65	0,0033	0,6910

A Tabela 4 mostra que dez termos entraram, na ordem de sua contribuição incremental, e nenhum foi removido. A área entra primeiro e, sozinha, produz $R^2_{\text{aj}} = 0{,}5299$; cidade e mobília elevam esse valor para 0,6444. **Quartos foi a única candidata não selecionada**: com os demais termos no modelo, o teste F parcial para sua inclusão resulta em $p = 0{,}9568$. Esse resultado é coerente com a Figura 5: dadas a área e o número de banheiros, o número de quartos não acrescenta informação sobre o aluguel, pois, para uma área fixa, mais quartos significam apenas cômodos menores.

Tabela 5: Comparação entre o modelo com todas as candidatas e o modelo selecionado.

Modelo	Termos	Parâmetros	R^2	R^2_{aj}	AIC	BIC
Completo	11	15	0,6914	0,6909	-4.908,20	-4.799,57
Selecionado	10	14	0,6914	0,6910	-4.910,20	-4.808,81
Teste F parcial para incluir Quartos: $F = 0,0029$; $p = 0,9568$.						

Os critérios de informação concordam com o *stepwise*: AIC e BIC são menores no modelo selecionado (Tabela 5), e a remoção de quartos não altera o R^2 até a quarta casa decimal.

Com mais de dez mil observações, porém, efeitos muito pequenos já são estatisticamente significativos: Animais entrou com $p = 0{,}0033$, mas elevou o R^2_{aj} em apenas 0,0002. Para avaliar a relevância prática de cada termo, a Tabela 6 apresenta a queda no R^2 quando o termo é retirado do modelo selecionado. Área, cidade e mobília dominam; os outros sete termos, isoladamente, contribuem com menos de 0,01 cada. A soma das contribuições é menor que o R^2 total porque parte da variabilidade explicada é compartilhada entre preditoras correlacionadas. O modelo selecionado foi mantido como modelo final, conforme o critério estabelecido, mas a interpretação dos efeitos considera também sua magnitude.

Tabela 6: Contribuição de cada termo: redução do R^2 ao retirá-lo do modelo selecionado.

Termo	gl	ΔR^2 ao remover	% do R^2
$\log_{10}$(Área)	1	0,0495	7,2
Cidade	4	0,0377	5,5
Mobília	1	0,0300	4,3
Banheiros	1	0,0078	1,1
BC(IPTU)	1	0,0066	1,0
Andar	1	0,0061	0,9
Vagas	1	0,0026	0,4
BC(Condomínio)	1	0,0018	0,3
Tipo	1	0,0012	0,2
Animais	1	0,0003	0,0

5 Resultados e discussões

5.1 Coeficientes do modelo selecionado

O modelo final foi ajustado por mínimos quadrados ordinários (OLS), com estimativa clássica, não robusta, da matriz de covariância dos coeficientes (Bloco 7). Os intervalos dos coeficientes foram calculados por $\widehat{\beta}_j \pm t_{0,975;\,n-p}\,\text{EP}(\widehat{\beta}_j)$, com $p = 14$ parâmetros. Como na Atividade 1, os p-valores e os intervalos são **nominais**: sua validade depende das premissas avaliadas nas próximas subseções.

Bloco 7: Ajuste do modelo final e efeitos na escala original.

```
modelo = final
print(modelo.summary())
```

```
ic = modelo.conf_int()
coeficientes = pd.DataFrame({"coef": modelo.params, "ep": modelo.bse, "t": modelo.
    tvalues,
                             "p": modelo.pvalues, "ic_inf": ic[0], "ic_sup": ic[1]})
print(coeficientes.round(5))
print(f"R2 = {modelo.rsquared:.4f} | R2 ajustado = {modelo.rsquared_adj:.4f} | "
      f"F({modelo.df_model:.0f}, {modelo.df_resid:.0f}) = {modelo.fvalue:.2f} | p = {
    modelo.f_pvalue:.3g}")


def variacao(b):
    """Variação percentual na mediana condicional do aluguel para um acréscimo b em
    log10."""
    return 100 * (10 ** np.asarray(b) - 1)


# Efeitos na escala original: categorias e contagens (+1 unidade), área (+10%),
# condomínio e IPTU (do 1.o ao 3.o quartil da variável original)
efeitos = {}
for nome in modelo.params.index.drop("Intercept"):
    escala = 1.0
    if nome == "log_area":
        escala = np.log10(1.10)
    elif nome in ("bc_cond", "bc_iptu"):
        original, lam = ("condominio", LAMBDA_COND) if nome == "bc_cond" else ("iptu",
     LAMBDA_IPTU)
        q1, q3 = df[original].quantile([0.25, 0.75])
        escala = stats.boxcox(np.array([q3 + 1.0]), lam)[0] - stats.boxcox(np.array([
    q1 + 1.0]), lam)[0]
    efeitos[nome] = dict(escala=escala, efeito=variacao(modelo.params[nome] * escala),
                         ic_inf=variacao(ic.loc[nome, 0] * escala), ic_sup=variacao(ic.
    loc[nome, 1] * escala))
efeitos = pd.DataFrame(efeitos).T
print(efeitos.round(2))
print(df[["condominio", "iptu"]].quantile([0.25, 0.75]))
```

Tabela 7: Coeficientes do modelo final, com IC nominais de 95%. Resposta: $\log_{10}$(Aluguel); referências: São Paulo, Apartamento, Não mobiliado e Aceita.

Termo	Estimativa	EP	t	p-valor	IC 95%
Intercepto	2,31109	0,01737	133,062	$< 0,0001$	[2,27704; 2,34513]
Cidade: Rio de Janeiro	-0,03571	0,00593	-6,022	$< 0,0001$	[-0,04733; -0,02408]
Cidade: Belo Horizonte	-0,11683	0,00627	-18,648	$< 0,0001$	[-0,12911; -0,10455]
Cidade: Porto Alegre	-0,14509	0,00640	-22,666	$< 0,0001$	[-0,15764; -0,13254]
Cidade: Campinas	-0,20240	0,00721	-28,085	$< 0,0001$	[-0,21653; -0,18828]
Tipo: Casa	0,05469	0,00860	6,356	$< 0,0001$	[0,03782; 0,07155]
Mobília: Mobiliado	0,14230	0,00450	31,651	$< 0,0001$	[0,13349; 0,15112]
Animais: Não aceita	0,01373	0,00467	2,941	0,0033	[0,00458; 0,02288]
$\log_{10}$(Área)	0,44291	0,01089	40,665	$< 0,0001$	[0,42156; 0,46426]
Banheiros	0,03838	0,00237	16,163	$< 0,0001$	[0,03373; 0,04304]
Vagas	0,01687	0,00181	9,340	$< 0,0001$	[0,01333; 0,02042]
Andar	0,00616	0,00043	14,237	$< 0,0001$	[0,00531; 0,00701]
BC(Condomínio)	0,00271	0,00035	7,673	$< 0,0001$	[0,00202; 0,00340]
BC(IPTU)	0,00871	0,00059	14,854	$< 0,0001$	[0,00756; 0,00985]

O modelo ajustado é

$$\begin{aligned}\widehat{\log_{10}(\mathit{aluguel})} = {} & 2{,}3111 - 0{,}0357\,\mathrm{RJ} - 0{,}1168\,\mathrm{BH} - 0{,}1451\,\mathrm{POA} - 0{,}2024\,\mathrm{CPS} \\ & + 0{,}0547\,\mathrm{Casa} + 0{,}1423\,\mathrm{Mobiliado} + 0{,}0137\,\mathrm{NãoAceita} + 0{,}4429\log_{10}(\mathit{area}) \\ & + 0{,}0384\,\mathit{banheiros} + 0{,}0169\,\mathit{vagas} + 0{,}0062\,\mathit{andar} \\ & + 0{,}0027\,\mathrm{BC}(\mathit{condominio}) + 0{,}0087\,\mathrm{BC}(\mathit{iptu}),\end{aligned}$$

em que RJ, BH, POA, CPS, Casa, Mobiliado e NãoAceita são variáveis indicadoras e BC$(\cdot)$ denota a transformação Box-Cox de $X + 1$ com os λ^* da Tabela 3. Todos os coeficientes têm IC de 95% que não contém zero (Tabela 7).

Como a resposta está em $\log_{10}$, um acréscimo b no preditor linear corresponde a multiplicar o aluguel por 10^b. Sob erros normais na escala logarítmica, $10^{\widehat{y}}$ estima a **mediana condicional** do aluguel, e não diretamente sua média, assim como na Atividade 1. A Tabela 8 converte os coeficientes em variações percentuais dessa mediana, mantidas as demais variáveis constantes.

Tabela 8: Efeitos estimados na escala original: variação percentual da mediana condicional do aluguel, com as demais variáveis constantes.

Termo	Comparação	Variação (%)	IC 95% (%)
Cidade: Rio de Janeiro	vs. referência	-7,9	[-10,3; -5,4]
Cidade: Belo Horizonte	vs. referência	-23,6	[-25,7; -21,4]
Cidade: Porto Alegre	vs. referência	-28,4	[-30,4; -26,3]
Cidade: Campinas	vs. referência	-37,3	[-39,3; -35,2]
Tipo: Casa	vs. referência	13,4	[9,1; 17,9]
Mobília: Mobiliado	vs. referência	38,8	[36,0; 41,6]
Animais: Não aceita	vs. referência	3,2	[1,1; 5,4]
$\log_{10}$(Área)	+10% de área	4,3	[4,1; 4,5]
Banheiros	+1 banheiro	9,2	[8,1; 10,4]
Vagas	+1 vaga	4,0	[3,1; 4,8]
Andar	+1 andar	1,4	[1,2; 1,6]
BC(Condomínio)	1.º → 3.º quartil	7,1	[5,2; 9,0]
BC(IPTU)	1.º → 3.º quartil	11,6	[10,0; 13,3]

- **Cidade:** para imóveis com as mesmas características, o aluguel mediano é 7,9% menor no Rio de Janeiro do que em São Paulo, 23,6% menor em Belo Horizonte, 28,4% menor em Porto Alegre e 37,3% menor em Campinas.
- **Mobília:** imóveis mobiliados têm aluguel mediano 38,8% maior, o maior efeito entre as variáveis binárias.
- **Área:** como resposta e preditora estão em $\log_{10}$, o coeficiente 0,443 é uma elasticidade. Um aumento de 10% na área está associado a um aluguel 4,3% maior, e dobrar a área multiplica o aluguel mediano por $2^{0,443} \approx 1{,}36$. O aluguel cresce menos que proporcionalmente à área; portanto, o valor por metro quadrado diminui à medida que o imóvel aumenta.
- **Banheiros, vagas e andar:** cada banheiro adicional está associado a um aumento de 9,2%, cada vaga a 4,0%, e cada andar a 1,4%, o que corresponde a cerca de 15% entre o 1.º e o 11.º andar.
- **Tipo:** a estimativa de 13,4% compara uma casa com um apartamento hipotético no andar 0. Em relação a um apartamento no 1.º andar, a diferença é $10^{0,0547-0,0062} - 1 \approx 11{,}8\%$.
- **Condomínio e IPTU:** como as escalas Box-Cox não têm interpretação direta, comparou-se o 1.º com o 3.º quartil de cada variável original. Passar de um condomínio de R$ 180 para R$ 1.283 está associado a um aluguel 7,1% maior; passar de um IPTU de R$ 40 para R$ 386, a um aluguel 11,6% maior. Como o IPTU é calculado sobre o valor venal do imóvel, essas variáveis provavelmente capturam parte do efeito da localização e do padrão do imóvel, que não estão na base.
- **Animais:** não aceitar animais está associado a um aluguel 3,2% maior. O efeito é estatisticamente significativo, mas pequeno, e pode refletir características não observadas, como edifícios de padrão mais alto com regras mais restritivas.

O intercepto corresponde a um imóvel de 1 m^2, sem banheiros, vagas, condomínio ou IPTU, situação fora da faixa observada; por isso, não tem interpretação direta. Os efeitos descritos são **associações condicionais** em dados observacionais, e não efeitos causais.

5.2 Coeficiente de determinação e teste F

Tabela 9: ANOVA do modelo final.

Fonte	gl	SQ	QM	F	p-valor
Regressão	13	838,780	64,5215	1.775,83	$< 0,0001$
Resíduos	10.306	374,449	0,03633	—	—
Total	10.319	1.213,228	—	—	—

$S = 0,1906$; $R^2 = 0,6914$; $R^2_{\text{aj}} = 0,6910$.

Tabela 10: Resumo OLS do modelo final.

Estatística	Valor	Estatística	Valor
Observações	10.320	gl modelo/resíduos	13/10.306
R^2	0,6914	R^2_{aj}	0,6910
F	1.775,83	$p(F)$	$< 10^{-300}$
Log-verossimilhança	2.469,098	AIC	-4.910,197
BIC	-4.808,811	Número de condição	227,9
Omnibus	159,097	p(Omnibus)	$2{,}835 \times 10^{-35}$
JB	270,283	p(JB)	$2{,}036 \times 10^{-59}$
Assimetria	0,121	Curtose	3,755
DW (ordem do arquivo)	2,036	Covariância	Clássica

O modelo explica **69,14%** da variabilidade amostral de $\log_{10}$(Aluguel) ($R^2 = 0{,}6914$), com $R^2_{\text{aj}} = 0{,}6910$ (Tabelas 9 e 10). Os dois valores são praticamente iguais porque n é muito maior que o número de parâmetros. Essa porcentagem se refere ao logaritmo do aluguel, não ao aluguel em reais.

O teste F de significância global resulta em $F(13;\ 10.306) = 1.775{,}83$, com $p < 10^{-300}$. Rejeita-se, portanto, a hipótese nula de que todos os coeficientes, exceto o intercepto, sejam nulos, ressalvada a natureza nominal do teste. O erro padrão residual é $S = 0{,}1906$ na escala $\log_{10}$. Sob normalidade, cerca de 68% dos aluguéis estariam entre $10^{-S} \approx 0{,}64$ e $10^{S} \approx 1{,}55$ vezes a mediana prevista, isto é, de 36% abaixo a 55% acima. Essa largura mostra que, embora o modelo explique uma fração importante da variabilidade, o erro de previsão para um imóvel individual permanece considerável.

5.3 Gráficos dos resíduos

O Bloco 8 gera os gráficos de diagnóstico e resume os resíduos por cidade e por decil dos valores preditos.

Bloco 8: Gráficos dos resíduos e resíduos por faixa de valores preditos.

```
gr.diagnostico(modelo, "diagnostico_final")
gr.residuos_preditores(
    modelo, df,
```

```
    {t: ROTULOS[t] for t in ["log_area", "banheiros", "vagas", "andar", "bc_cond", "
    bc_iptu"] if t in selecionados},
    {t: CATEGORICAS[t] for t in ["cidade", "tipo", "mobilia"] if t in selecionados},
    "residuos_preditores")

residuos = modelo.resid
print(residuos.describe().round(4))
print(f"Assimetria = {stats.skew(residuos):.3f} | curtose = {stats.kurtosis(residuos,
    fisher=False):.3f}")
print(f"Fração com |resíduo| > 2S: {np.mean(np.abs(residuos) > 2 * np.sqrt(modelo.
    scale)):.4f}")
print(df.assign(residuo=residuos).groupby("cidade", observed=True)["residuo"].agg(["
    count", "mean", "std"]).round(4))

# Concentração de anúncios no valor máximo usual (possível teto da coleta)
print(f"Aluguel = R$ 15.000: {(df['aluguel'] == 15_000).sum()} | acima de R$ 15.000:
    {(df['aluguel'] > 15_000).sum()}")
decis = df.assign(residuo=residuos, predito=modelo.fittedvalues, teto=df["aluguel"] >=
     15_000,
                  decil=pd.qcut(modelo.fittedvalues, 10, labels=range(1, 11)))
por_decil = decis.groupby("decil", observed=True).agg(
    predito_medio=("predito", "mean"), media_residuo=("residuo", "mean"),
    dp_residuo=("residuo", "std"), prop_teto=("teto", "mean"))
print(por_decil.round(4))
```

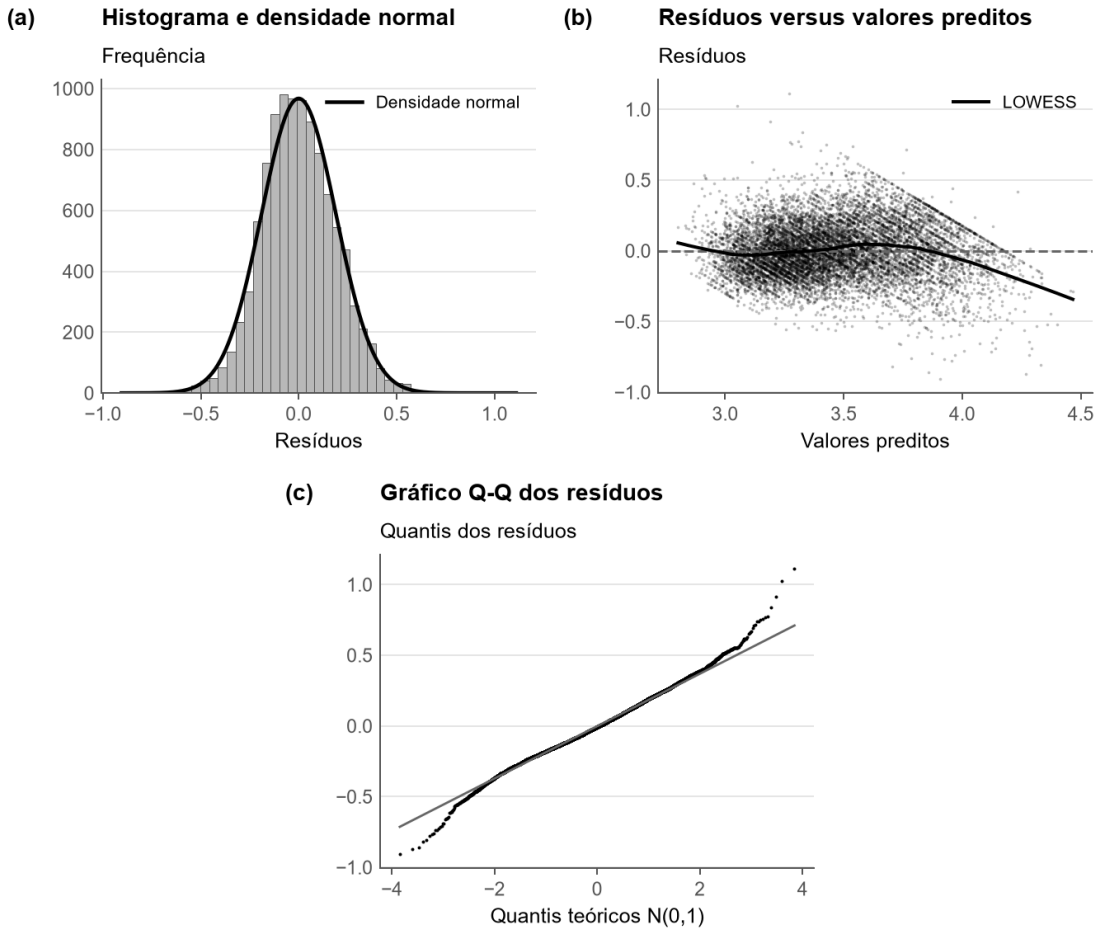


Figura 6: Diagnóstico do modelo final: (a) histograma com densidade normal de referência; (b) resíduos versus valores preditos, com curva LOWESS; (c) gráfico Q–Q.

Histograma e Q–Q. O histograma (Figura 6a) é unimodal, aproximadamente simétrico e acompanha bem a densidade normal, com pico um pouco mais alto e caudas um pouco mais pesadas (assimetria de 0,121 e curtose de 3,755; na normal, 0 e 3). No Q–Q (Figura 6c), os pontos seguem a reta de referência na região central e se afastam dela nas duas caudas, sobretudo na superior. O maior resíduo, 1,11, corresponde a um aluguel observado cerca de 13 vezes maior que a mediana prevista. A fração de resíduos com $|e_i| > 2S$ é de 4,48%, próxima dos 4,55% esperados sob normalidade.

Resíduos versus preditos. Na Figura 6b, a nuvem está centrada em zero em quase toda a faixa de valores preditos, sem um funil pronunciado. As faixas diagonais decorrem do arredondamento dos aluguéis (R$ 1.500, R$ 2.000 etc.): para um valor observado fixo y, o resíduo $y - \widehat{y}$ diminui linearmente com $\widehat{y}$. Trata-se de um efeito da discretização da resposta, e não de uma falha do modelo. A faixa mais visível, que forma a borda superior da nuvem à direita, corresponde ao aluguel de R$ 15.000. A curva LOWESS fica levemente abaixo de zero entre cerca de 3,0 e 3,3, levemente acima entre 3,5 e 3,8 e decresce a partir de cerca de 3,9, isto é, de aproximadamente R$ 8 mil.

Tabela 11: Resíduos por decil dos valores preditos.

Decil	Predito médio	10^{predito} (R$)	Média res.	DP res.	% com aluguel $\geq$ R$ 15 mil
1	3,037	1.089	-0,0222	0,1659	0,0
2	3,162	1.451	-0,0228	0,1636	0,0
3	3,239	1.734	-0,0180	0,1669	0,1
4	3,311	2.045	-0,0075	0,1754	0,0
5	3,385	2.427	0,0031	0,1803	0,0
6	3,466	2.924	0,0197	0,1870	0,3
7	3,556	3.594	0,0470	0,1957	0,6
8	3,662	4.596	0,0420	0,2050	1,6
9	3,792	6.196	0,0251	0,2151	5,1
10	3,995	9.895	-0,0664	0,2131	15,8

A Tabela 11 quantifica esse padrão. A média dos resíduos é de cerca de $-0{,}02$ nos três primeiros decis, chega a $+0{,}047$ no sétimo e cai para $-0{,}066$ no décimo, decil em que 15,8% dos anúncios têm aluguel de R$ 15.000 ou mais. Nesse decil, o aluguel observado é, em média, cerca de 14% menor que a mediana prevista ($10^{-0{,}066} \approx 0{,}86$). A explicação mais provável é o **teto aparente da coleta**: para imóveis grandes e caros, o aluguel observado fica limitado a cerca de R$ 15.000, enquanto o modelo prolonga a tendência linear. Isso produz um desvio sistemático de L no extremo superior. Nos demais decis, os desvios de linearidade são pequenos. O desvio padrão dos resíduos cresce de 0,164 nos primeiros decis para 0,215 no nono, o que corresponde a uma variância cerca de 1,7 vez maior nas faixas superiores; esse resultado é retomado na avaliação de E.

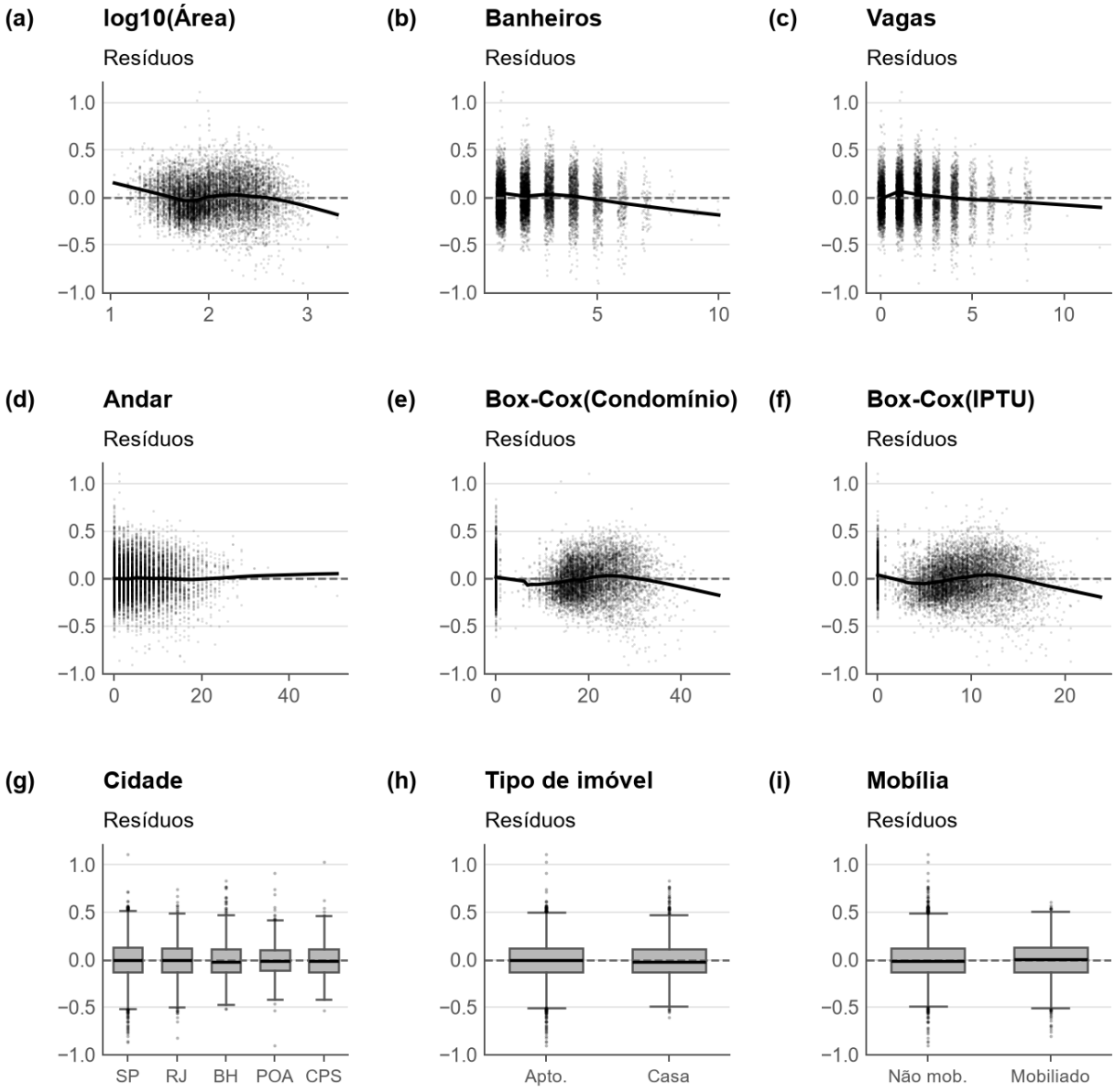


Figura 7: Resíduos versus cada preditora selecionada, com curva LOWESS para as numéricas (a)–(f) e boxplots para as categóricas (g)–(i). SP: São Paulo; RJ: Rio de Janeiro; BH: Belo Horizonte; POA: Porto Alegre; CPS: Campinas.

Resíduos versus preditoras. Nos gráficos contra cada preditora (Figura 7), as curvas LOWESS permanecem próximas de zero na região em que se concentra a grande maioria dos registros, o que dá suporte à forma linear adotada para cada variável. Os desvios aparecem nos extremos, onde há poucos dados: imóveis pequenos (abaixo de cerca de 40 m^2, isto é, $\log_{10}(\text{Área}) < 1{,}6$) têm resíduos positivos, compatíveis com um preço por metro quadrado maior em quitinetes e estúdios; imóveis muito grandes, com muitos banheiros ou com condomínio e IPTU altos têm resíduos negativos, o mesmo padrão associado ao teto de R$ 15.000. Nas variáveis categóricas, a média dos resíduos é zero em todas as categorias por construção, pois cada categoria tem sua própria indicadora, mas a dispersão varia: o desvio padrão dos resíduos é de 0,164 em Porto Alegre e de 0,174 em Campinas, contra 0,197 em São Paulo.

5.4 Testes estatísticos sobre os resíduos

O Bloco 9 aplica cinco testes de normalidade e três testes de homogeneidade das variâncias. No teste de Goldfeld–Quandt, os registros foram ordenados pelos valores preditos, e os 20% centrais foram descartados, comparando-se as variâncias residuais dos 40% inferiores e dos 40% superiores. O teste de Brown–Forsythe, versão do teste de Levene baseada na mediana, compara as variâncias dos resíduos entre as cidades.

Bloco 9: Testes de normalidade e de homogeneidade das variâncias dos resíduos.

```
from statsmodels.stats.diagnostic import het_breuschpagan, het_goldfeldquandt,
    lilliefors, normal_ad
from statsmodels.stats.stattools import durbin_watson

r = modelo.resid.to_numpy()
X = modelo.model.exog

normalidade = {
    "Shapiro-Wilk": stats.shapiro(r)[:2],
    "Kolmogorov-Smirnov (Lilliefors)": lilliefors(r, dist="norm"),
    "Anderson-Darling": normal_ad(r),
    "Jarque-Bera": tuple(stats.jarque_bera(r)),
    "D'Agostino-Pearson (Omnibus)": tuple(stats.normaltest(r)),
}

ordem = np.argsort(modelo.fittedvalues.to_numpy())  # Goldfeld-Quandt: ordenação pelos
     preditos
grupos_cidade = [r[(df["cidade"] == c).to_numpy()] for c in df["cidade"].cat.
    categories]
variancia = {
    "Breusch-Pagan (Koenker)": het_breuschpagan(r, X)[:2],
    "Goldfeld-Quandt (preditos)": het_goldfeldquandt(modelo.model.endog[ordem], X[
    ordem], drop=0.2,
                                                     result_object=False)[:2],
    "Brown-Forsythe (cidades)": tuple(stats.levene(*grupos_cidade, center="median")),
}

testes = pd.DataFrame(
    [dict(grupo="Normalidade", teste=k, estatistica=v[0], p=v[1]) for k, v in
    normalidade.items()]
    + [dict(grupo="Variância", teste=k, estatistica=v[0], p=v[1]) for k, v in
    variancia.items()])
print(testes)
print(f"Durbin-Watson (ordem do arquivo) = {durbin_watson(r):.3f}")
```

Tabela 12: Testes sobre os resíduos do modelo final ($n = 10.320$).

Premissa	Teste	Estatística	p-valor
Normalidade	Shapiro–Wilk	0,9948	$7{,}267 \times 10^{-19}$
Normalidade	Kolmogorov–Smirnov (Lilliefors)	0,0212	$\leq 0,001$
Normalidade	Anderson–Darling	9,0315	$6{,}709 \times 10^{-22}$
Normalidade	Jarque–Bera	270,2826	$2{,}036 \times 10^{-59}$
Normalidade	D'Agostino–Pearson (Omnibus)	159,0967	$2{,}835 \times 10^{-35}$
Variância	Breusch–Pagan (Koenker)	216,2812	$6{,}100 \times 10^{-39}$
Variância	Goldfeld–Quandt (preditos)	1,5634	$3{,}237 \times 10^{-42}$
Variância	Brown–Forsythe (cidades)	15,8176	$6{,}517 \times 10^{-13}$

Normalidade. Todos os testes rejeitam a normalidade dos resíduos (Tabela 12). Com $n = 10.320$, porém, esses testes detectam desvios muito pequenos; o resultado deve ser lido em conjunto com a magnitude do desvio, que é pequena: assimetria de 0,121, curtose de 3,755 e uma fração de resíduos acima de $2S$ praticamente igual à esperada. O desvio concentra-se nas caudas, como mostrou o Q–Q. A SciPy alerta que o p-valor de Shapiro–Wilk pode ser impreciso para $n > 5.000$, por isso ele é apresentado apenas como complemento; o p-valor de Lilliefors é obtido de uma tabela e truncado em 0,001. Conclui-se que N não é estritamente satisfeita, mas a violação é leve. Para a inferência sobre os coeficientes, com esse tamanho amostral, o teorema central do limite torna os intervalos aproximadamente válidos; para intervalos de predição de aluguéis individuais, as caudas mais pesadas podem comprometer a cobertura nominal nos extremos.

Homogeneidade das variâncias. Os três testes rejeitam a homocedasticidade. Na estatística de Goldfeld–Quandt, $F = 1{,}563$ indica que a variância residual nos 40% maiores valores preditos é 56% maior que nos 40% menores, e o teste de Brown–Forsythe confirma variâncias diferentes entre as cidades. Assim, E **não é satisfeita**; a heterocedasticidade é moderada, sem o funil acentuado do modelo na escala original (Seção 5.6). Sua consequência sobre os erros padrão é avaliada a seguir.

Independência. A estatística de Durbin–Watson calculada na ordem do arquivo é 2,036. Como os dados são transversais e a ordem do arquivo não corresponde a uma sequência temporal de coleta, esse valor não é usado para declarar independência.

5.5 Multicolinearidade, observações influentes e erros padrão robustos

O Bloco 10 calcula os fatores de inflação da variância (VIF), as medidas de influência e os erros padrão robustos à heterocedasticidade do tipo HC3 [7, 8].

Bloco 10: VIF, observações influentes e erros padrão robustos (HC3).

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Multicolinearidade
```

```
Xd = pd.DataFrame(modelo.model.exog, columns=modelo.model.exog_names)
vif = pd.Series({c: variance_inflation_factor(Xd.values, i)
                 for i, c in enumerate(Xd.columns) if c != "Intercept"})
print(vif.round(2))
print(f"Número de condição = {modelo.condition_number:.1f}")

# Observações influentes
infl = modelo.get_influence()
n, p = int(modelo.nobs), int(modelo.df_model) + 1
diag = df.assign(cook=infl.cooks_distance[0], alav=infl.hat_matrix_diag,
                 rstud=infl.resid_studentized_external)
print(f"max D = {diag['cook'].max():.4f} | D > 4/n: {(diag['cook'] > 4 / n).sum()} | "
      f"h > 2p/n: {(diag['alav'] > 2 * p / n).sum()} | |rstud| > 3: {(diag['rstud'].
    abs() > 3).sum()}")
print(diag.nlargest(5, "cook")[["cidade", "tipo", "area", "quartos", "banheiros", "
    andar",
                                "condominio", "aluguel", "iptu", "cook", "alav", "
    rstud"]])
gr.influencia(modelo, "influencia")

# Sensibilidade: reajuste sem os registros com D > 4/n (somente para comparação)
sem_influentes = smf.ols(modelo.model.formula, data=df[diag["cook"] <= 4 / n]).fit()
desvio = ((sem_influentes.params - modelo.params) / modelo.bse).abs()
print(f"R2 sem influentes = {sem_influentes.rsquared:.4f} | maior mudança = {desvio.
    max():.2f} EP ({desvio.idxmax()})")
print(pd.DataFrame({"todos": modelo.params, "sem_influentes": sem_influentes.params}).
    round(4))

# Erros padrão robustos à heterocedasticidade (HC3)
robusto = smf.ols(modelo.model.formula, data=df).fit(cov_type="HC3")
comparacao_ep = pd.DataFrame({"ep_classico": modelo.bse, "ep_hc3": robusto.bse,
                              "razao": robusto.bse / modelo.bse,
                              "p_classico": modelo.pvalues, "p_hc3": robusto.pvalues})
print(comparacao_ep.round(5))
```

Tabela 13: VIF e comparação entre os erros padrão clássicos e robustos (HC3).

Termo	VIF	EP clássico	EP HC3	Razão HC3/clássico	p (HC3)
Intercepto	—	0,01737	0,01854	1,067	$< 0,0001$
Cidade: Rio de Janeiro	1,19	0,00593	0,00608	1,026	$< 0,0001$
Cidade: Belo Horizonte	1,15	0,00627	0,00634	1,012	$< 0,0001$
Cidade: Porto Alegre	1,16	0,00640	0,00577	0,902	$< 0,0001$
Cidade: Campinas	1,08	0,00721	0,00668	0,927	$< 0,0001$
Tipo: Casa	3,72	0,00860	0,00844	0,980	$< 0,0001$
Mobília: Mobiliado	1,06	0,00450	0,00459	1,021	$< 0,0001$
Animais: Não aceita	1,06	0,00467	0,00470	1,007	0,0035
$\log_{10}$(Área)	3,96	0,01089	0,01168	1,072	$< 0,0001$
Banheiros	3,20	0,00237	0,00252	1,062	$< 0,0001$
Vagas	2,37	0,00181	0,00199	1,101	$< 0,0001$
Andar	1,51	0,00043	0,00045	1,042	$< 0,0001$
BC(Condomínio)	3,48	0,00035	0,00037	1,054	$< 0,0001$
BC(IPTU)	2,00	0,00059	0,00063	1,072	$< 0,0001$

Multicolinearidade. O maior VIF é 3,96, de $\log_{10}$(Área), e nenhum valor ultrapassa 5, limite usual de alerta (Tabela 13). A exclusão de quartos, que tinha correlação de 0,81 com a área, contribuiu para esse resultado. A multicolinearidade remanescente é moderada e não compromete a estabilidade das estimativas, cujos erros padrão são pequenos em relação aos coeficientes.

Observações influentes. A maior distância de Cook é $D = 0{,}0051$, muito abaixo dos limites usuais de 0,5 e 1; nenhum registro, isoladamente, determina o ajuste (Figura 8). A regra $D > 4/n$ marca 530 registros (5,1%), mas tende a sinalizar muitos pontos quando n é grande. Há 56 resíduos estudentizados com $|r_i^*| > 3$, cerca do dobro dos 28 esperados sob normalidade, o que é coerente com as caudas mais pesadas.

Tabela 14: Cinco registros com maior distância de Cook (D), com alavancagem h e resíduo estudentizado r^*.

Cidade	Tipo	Área	Banh.	Andar	Condomínio	Aluguel	IPTU	D	h	r^*
São Paulo	Apto.	850	7	1	14.000	4.000	0	0,0051	0,0082	-2,94
Campinas	Apto.	75	1	1	296	12.000	0	0,0044	0,0021	5,40
Porto Alegre	Apto.	900	5	4	2.700	1.000	542	0,0042	0,0026	-4,76
São Paulo	Apto.	717	5	16	4.300	2.500	5.032	0,0038	0,0025	-4,59
São Paulo	Casa	1.600	5	0	1	6.900	5.000	0,0034	0,0062	-2,74

Os registros mais influentes (Tabela 14) são suspeitos. Por exemplo, há um apartamento de 75 m^2 em Campinas alugado por R$ 12.000 e um de 900 m^2 em Porto Alegre por R$ 1.000, o que sugere erros de digitação na área ou no aluguel. Esses registros, porém, não atendem aos critérios objetivos de implausibilidade da Seção 2.3 e foram mantidos; sua verificação exigiria acesso aos anúncios originais. Como análise de sensibilidade, o modelo foi reajustado sem os 530 registros com $D > 4/n$. Todos os coeficientes mantiveram o sinal e a ordem de grandeza; a maior alteração ocorreu em BC(Condomínio), de 0,0027 para 0,0038, ou 3,1 erros padrão. O R^2 desse reajuste sobe para 0,7605, mas isso ocorre por construção, pois foram retirados justamente os registros pior ajustados, e não indica melhora do modelo.

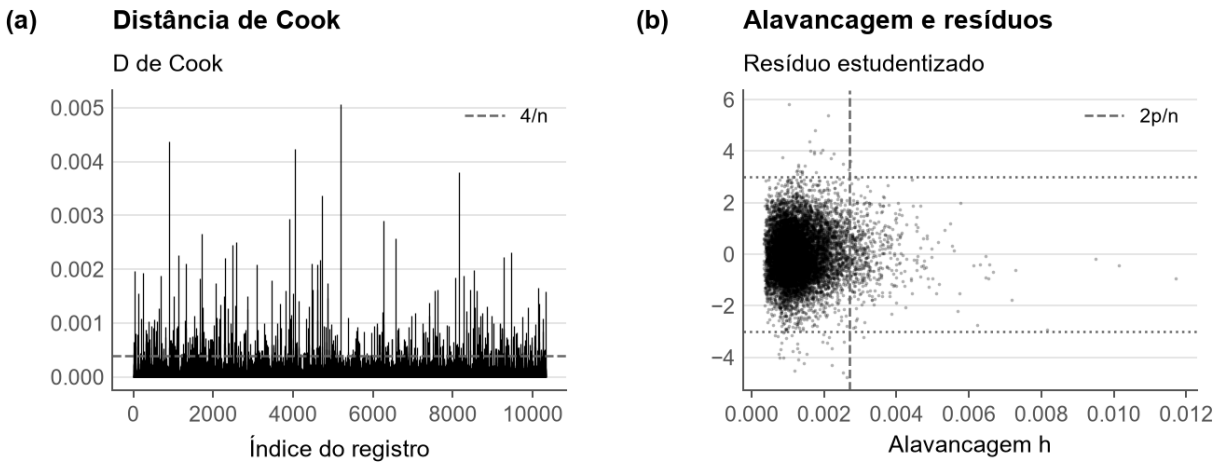


Figura 8: (a) Distância de Cook por registro, com a referência $4/n$; (b) alavancagem versus resíduo estudentizado, com as referências $2p/n$ e ± 3.

Erros padrão robustos. Diante da heterocedasticidade detectada, os erros padrão foram recalculados com o estimador HC3, que não pressupõe variância constante. As razões entre os erros robustos e os clássicos variam de 0,90 a 1,10 (Tabela 13), e todos os coeficientes continuam significativos. O de Animais, o menor efeito, tem $p = 0{,}0035$. Portanto, **as conclusões sobre os coeficientes são robustas à violação de E**. O mesmo não vale automaticamente para os intervalos de predição individuais, cuja largura deveria variar com o valor previsto.

5.6 Comparação com a escala original e desempenho fora da amostra

O Bloco 11 ajusta, como referência, um modelo com as mesmas 11 candidatas sem nenhuma transformação e compara os dois modelos por validação cruzada em dez partes. Em cada rodada, os modelos são ajustados em 90% dos registros e avaliados nos 10% restantes. As previsões do modelo final foram retransformadas para reais com a correção de *smearing* de Duan [6], que estima a média condicional, e não a mediana.

Bloco 11: Modelo sem transformações e validação cruzada.

```
# Modelo de referência: mesmas preditoras candidatas, sem nenhuma transformação
formula_original = ("aluguel ~ cidade + tipo + mobilia + animal + area + quartos +
    banheiros"
                    " + vagas + andar + condominio + iptu")
original = smf.ols(formula_original, data=df).fit()
print(f"Escala original: R2 = {original.rsquared:.4f} | R2 ajustado = {original.
    rsquared_adj:.4f} | "
      f"assimetria = {stats.skew(original.resid):.3f} | "
      f"curtose = {stats.kurtosis(original.resid, fisher=False):.3f}")
gr.comparacao_escalas(original, modelo, "comparacao_escalas")

# Validação cruzada em 10 partes: previsões sempre feitas em registros fora do ajuste
dobras = np.random.default_rng(2026).permutation(len(df)) % 10
previsoes = pd.DataFrame(index=df.index, columns=["log_final", "rs_final", "
    rs_original"], dtype=float)
for k in range(10):
    treino, teste = df[dobras != k], df[dobras == k]
    ajuste = smf.ols(modelo.model.formula, data=treino).fit()
    smearing = np.mean(10 ** ajuste.resid)  # retransformação de Duan (1983)
    previsoes.loc[teste.index, "log_final"] = ajuste.predict(teste)
    previsoes.loc[teste.index, "rs_final"] = 10 ** ajuste.predict(teste) * smearing
    previsoes.loc[teste.index, "rs_original"] = smf.ols(formula_original, data=treino).
    fit().predict(teste)


def r2(y, y_hat):
    return 1 - np.sum((y - y_hat) ** 2) / np.sum((y - np.mean(y)) ** 2)


y, y_log = df["aluguel"], df["log_aluguel"]
validacao = pd.DataFrame({
    "Final (log10)": dict(R2_log=r2(y_log, previsoes["log_final"]),
                          RMSE_log=np.sqrt(np.mean((y_log - previsoes["log_final"]) **
     2)),
                          R2_reais=r2(y, previsoes["rs_final"]),
```

```
                          RMSE_reais=np.sqrt(np.mean((y - previsoes["rs_final"]) ** 2))
    ,
                          EAM_reais=np.mean(np.abs(y - previsoes["rs_final"])),
                          EPAM_mediano=100 * np.median(np.abs(y - previsoes["rs_final"
    ]) / y),
                          negativas=int((previsoes["rs_final"] <= 0).sum()),
    "Sem transformações": dict(R2_log=np.nan, RMSE_log=np.nan,
                               R2_reais=r2(y, previsoes["rs_original"]),
                               RMSE_reais=np.sqrt(np.mean((y - previsoes["rs_original"
    ]) ** 2)),
                               EAM_reais=np.mean(np.abs(y - previsoes["rs_original"])),

                               EPAM_mediano=100 * np.median(np.abs(y - previsoes["
    rs_original"]) / y),
                               negativas=int((previsoes["rs_original"] <= 0).sum())),
})
print(validacao.round(4))
print(f"Fator multiplicativo típico 10^S = {10 ** np.sqrt(modelo.scale):.3f}")
```

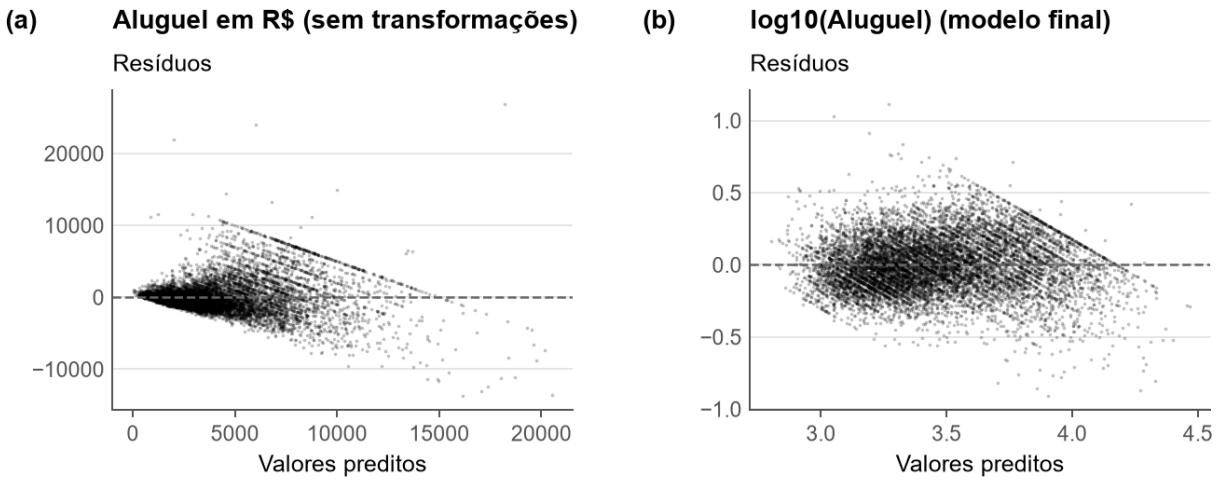


Figura 9: Resíduos versus valores preditos: (a) modelo com aluguel e preditoras em reais, sem transformações; (b) modelo final.

O modelo sem transformações tem R^2= 0,5902 (R^2_{aj}= 0,5897), valor que se refere à variabilidade do aluguel em reais e não pode ser comparado diretamente com o 0,6914 do modelo final. Seus resíduos, contudo, mostram problemas graves (Figura 9a): a variância cresce fortemente com o valor previsto (funil), a assimetria é de 1,345 e a curtose de 12,280. N e E são claramente violadas, e os intervalos e testes desse modelo não seriam confiáveis. A borda diagonal formada pelos aluguéis de R$ 15.000 também é visível. As transformações reduziram drasticamente esses problemas (Figura 9b).

Tabela 15: Desempenho preditivo por validação cruzada em dez partes (previsões sempre fora da amostra de ajuste).

Métrica (validação cruzada, 10 partes)	Modelo final	Sem transformações
R^2 fora da amostra, escala $\log_{10}$	0,6906	—
RMSE, escala $\log_{10}$	0,1907	—
R^2 fora da amostra, em R$	0,5112	0,5868
RMSE (R$)	2.399	2.206
Erro absoluto médio (R$)	1.458	1.412
Erro percentual absoluto mediano (%)	31,3	31,3
Previsões não positivas	0	0

Na escala logarítmica, o R^2 fora da amostra (0,6906) é praticamente igual ao do ajuste (0,6914), e o RMSE (0,1907) coincide com S; não há sobreajuste, como esperado quando n é muito maior que o número de parâmetros (Tabela 15). **Em reais, porém, o modelo sem transformações tem RMSE menor** (R$ 2.206 contra R$ 2.399), e os dois modelos têm erro absoluto médio semelhante e o mesmo erro percentual absoluto mediano, de 31,3%. Esse resultado não contradiz a escolha das transformações. O OLS na escala original minimiza justamente o erro quadrático em reais, métrica dominada pelos imóveis caros, nos quais os erros são maiores. O modelo logarítmico, por sua vez, modela erros relativos, produz resíduos muito mais próximos das premissas e permite inferência sobre os coeficientes, com erros padrão HC3, e interpretação em termos percentuais. Sua retransformação para a média em reais depende de uma correção que supõe variância constante, hipótese violada, e é afetada pelo teto de R$ 15.000 no extremo superior.

Em termos práticos, ambos os modelos erram tipicamente cerca de 31% do aluguel em uma previsão individual. O fator limitante não parece ser a forma funcional, mas a ausência de informações sobre a localização do imóvel dentro da cidade, como bairro ou coordenadas, que é reconhecidamente um dos principais determinantes do aluguel.

5.7 Síntese: qualidades, limitações e premissas LINE

Qualidade do ajuste. O modelo explica 69% da variabilidade de $\log_{10}$(Aluguel), com o mesmo desempenho fora da amostra, e seus coeficientes têm sinais e magnitudes plausíveis. Ele fornece um bom ajuste para descrever **relações médias**, como as diferenças entre cidades e os efeitos de área, mobília e banheiros. Para prever o aluguel de um imóvel específico, porém, o erro típico de cerca de 31% limita seu uso como ferramenta de avaliação individual.

Tabela 16: Síntese de LINE no modelo final. “Compatível” significa ausência de evidência forte de violação, e não comprovação da condição.

Premissa	Evidência	Avaliação
L: função média linear	LOWESS próximas de zero na região central dos preditos e de cada preditora; média dos resíduos de $-0{,}066$ no decil superior, associado ao teto de R$ 15.000; desvios em imóveis muito pequenos ou muito grandes	Compatível na maior parte dos dados; violação localizada no extremo superior
I: independência	Dados transversais; duplicatas exatas removidas; localização dentro da cidade ausente; possíveis anúncios do mesmo edifício ou bairro	Não verificável; risco de dependência espacial
N: normalidade	Histograma e Q–Q próximos da normal no centro; assimetria de 0,121 e curtose de 3,755; cinco testes rejeitam com $n = 10.320$	Violação leve, concentrada nas caudas
E: variância constante	Desvio padrão dos resíduos de 0,164 a 0,215 entre decis; Breusch–Pagan, Goldfeld–Quandt e Brown–Forsythe rejeitam	Não sustentada; heterocedasticidade moderada, contornada com EP HC3

A Tabela 16 resume as premissas, avaliadas conforme as Lições 4 e 7 do STAT 501 [4]. Em comparação com o modelo sem transformações, o modelo final representa uma melhora substancial em N e E e permite interpretar os efeitos em termos percentuais. As violações remanescentes têm consequências delimitadas: a heterocedasticidade não altera as conclusões sobre os coeficientes quando se usam erros padrão HC3; o desvio de normalidade é leve; e o desvio de linearidade concentra-se no extremo superior, onde a própria coleta parece limitar o aluguel observado. A independência, por sua vez, não pode ser confirmada com as informações disponíveis.

6 Conclusões

- **Pré-processamento:** a análise utilizou 10.320 anúncios, após a remoção de 358 duplicatas e de 14 registros implausíveis. Seguro e total foram excluídos por conterem o próprio aluguel, e foi criada a variável *tipo*. Aluguel e área foram transformados com $\log_{10}$, e condomínio e IPTU, com Box-Cox.
- **Seleção:** o *stepwise* com $\alpha_E = \alpha_R = 0{,}15$ manteve dez das 11 candidatas e excluiu apenas quartos, redundante diante da área e do número de banheiros. Os critérios AIC e BIC concordaram com essa escolha. Área, cidade e mobília concentram a maior parte do poder explicativo.
- **Efeitos:** com as demais características constantes, o aluguel mediano é de 8% a 37% menor nas outras cidades do que em São Paulo; aumenta 4,3% para cada 10% de área, 38,8% quando o imóvel é mobiliado, 9,2% por banheiro, 4,0% por vaga e 1,4% por andar.
- **Ajuste e premissas:** $R^2 = 0{,}6914$ na escala logarítmica, confirmado por validação cruzada, e teste F altamente significativo. N é aproximadamente atendida; E é violada de forma moderada, com conclusões preservadas pelos erros padrão HC3; L tem desvio no extremo superior, associado ao teto aparente de R$ 15.000; I não é verificável.

- **Limitações e extensões:** o erro típico de cerca de 31% em previsões individuais indica que falta informação relevante, principalmente a localização dentro da cidade. Possíveis extensões incluem obter bairro ou coordenadas, modelar o teto explicitamente com regressão censurada (Tobit), investigar interações, como cidade × área para valores por metro quadrado diferentes entre cidades, e modelar a variância por mínimos quadrados ponderados.

As ferramentas de inteligência artificial foram utilizadas como apoio à implementação do código, à análise estatística e à redação do relatório [10]; esse apoio não substitui a responsabilidade dos autores pela revisão, pela análise e pela interpretação.

Referências

[1] RUBENSSJR. *Brazilian houses to rent*. Kaggle, 2020. Disponível em: `https://www.kaggle.com/datasets/rubenssjr/brasilian-houses-to-rent`. Acesso em: 31 out. 2026.

[2] OpenML. *Brazilian_houses* (ID 42688, versão 4). Disponível em: `https://www.openml.org/d/42688`. Acesso em: 31 out. 2026.

[3] Pennsylvania State University. *STAT 501: Regression Methods – Lesson 10.2, Stepwise Regression*. Disponível em: `https://online.stat.psu.edu/stat501/lesson/10/10.2`. Acesso em: 31 out. 2026.

[4] Pennsylvania State University. *STAT 501: Regression Methods – Lessons 4 e 7, Model Assumptions*. Disponível em: `https://online.stat.psu.edu/stat501/`. Acesso em: 31 out. 2026.

[5] BOX, G. E. P.; COX, D. R. An analysis of transformations. *Journal of the Royal Statistical Society: Series B*, v. 26, n. 2, p. 211–252, 1964.

[6] DUAN, N. Smearing estimate: a nonparametric retransformation method. *Journal of the American Statistical Association*, v. 78, n. 383, p. 605–610, 1983.

[7] WHITE, H. A heteroskedasticity-consistent covariance matrix estimator and a direct test for heteroskedasticity. *Econometrica*, v. 48, n. 4, p. 817–838, 1980.

[8] MACKINNON, J. G.; WHITE, H. Some heteroskedasticity-consistent covariance matrix estimators with improved finite sample properties. *Journal of Econometrics*, v. 29, n. 3, p. 305–325, 1985.

[9] SEABOLD, S.; PERKTOLD, J. Statsmodels: econometric and statistical modeling with Python. In: *Proceedings of the 9th Python in Science Conference*, 2010.

[10] Anthropic. *Claude Code*. Assistente de inteligência artificial utilizado como apoio à implementação do código, à análise e à redação do relatório. Disponível em: `https://claude.com/claude-code`. Acesso em: 31 out. 2026.

A Funções auxiliares e execução dos blocos

O script `executar.py` executa os Blocos 1 a 11 em sequência, no mesmo ambiente, e grava a saída de cada bloco na pasta `saidas`. Ao final, executa `exportar_tabelas.py`, que apenas converte os resultados já calculados em tabelas LaTeX com formatação numérica brasileira e, por isso, não é reproduzido aqui. O módulo `graficos.py` contém as funções de visualização, que seguem o mesmo padrão gráfico em tons de cinza da Atividade 1. Para reproduzir a análise, basta executar `python codigo/executar.py` na pasta do projeto, com o arquivo `dados/Brazilian_houses.arff` obtido no OpenML.

codigo/executar.py

```
"""Executa os blocos em sequência, como células de um notebook, e grava a saída de cada um."""
import contextlib
import io
import os
import sys
import warnings
from pathlib import Path

CODIGO = Path(__file__).resolve().parent
RAIZ = CODIGO.parent
SAIDAS = RAIZ / "saidas"
SAIDAS.mkdir(exist_ok=True)
os.chdir(RAIZ)
sys.path.insert(0, str(CODIGO))

import pandas as pd

pd.set_option("display.width", 160)
pd.set_option("display.max_columns", 20)
warnings.filterwarnings("ignore", message=".*p-value may not be accurate.*")

ambiente = {"__name__": "__main__"}
for bloco in sorted(CODIGO.glob("b[0-9][0-9]_*.py")):
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):
        exec(compile(bloco.read_text(encoding="utf-8"), bloco.name, "exec"), ambiente)
    (SAIDAS / f"{bloco.stem}.txt").write_text(buffer.getvalue(), encoding="utf-8")
    print(f"=== {bloco.name} ===\n{buffer.getvalue()}")

exec(compile((CODIGO / "exportar_tabelas.py").read_text(encoding="utf-8"), "exportar_tabelas.py", "exec"),
     ambiente)
```

codigo/graficos.py

```
"""Funções auxiliares de visualização (mesmo padrão em tons de cinza da Atividade 1)."""
from __future__ import annotations

import math
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from PIL import Image
from scipy import stats
from statsmodels.nonparametric.smoothers_lowess import lowess

SAIDA = Path(__file__).resolve().parent.parent / "figuras"
SAIDA.mkdir(exist_ok=True)

PRETO = "#000000"
CINZA = "#555555"
GRADE = "#E5E5E5"
```

```
PREENCHIMENTO = "#B8B8B8"
PREENCHIMENTO_CLARO = "#E6E6E6"
REFERENCIA = "#666666"

plt.rcParams.update({
    "font.family": ["Arial", "DejaVu Sans"],
    "font.size": 10,
    "axes.edgecolor": CINZA,
    "axes.linewidth": 0.8,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "axes.grid.axis": "y",
    "grid.color": GRADE,
    "grid.linewidth": 0.8,
    "axes.axisbelow": True,
    "xtick.color": CINZA,
    "ytick.color": CINZA,
    "xtick.labelsize": 9,
    "ytick.labelsize": 9,
    "ytick.major.size": 0,
    "xtick.major.size": 3,
    "legend.frameon": False,
    "figure.facecolor": "white",
    "savefig.facecolor": "white",
})


def painel(ax, titulo: str, rotulo_y: str = "", rotulo_x: str = "", letra: str | None = None):
    """Título em negrito, rótulo do eixo y na horizontal e letra do painel, como na Atividade 1."""
    ax.annotate(titulo, (0, 1), xycoords="axes fraction", xytext=(0, 24), textcoords="offset points",
                fontsize=11, fontweight="bold", ha="left", va="bottom")
    if rotulo_y:
        ax.annotate(rotulo_y, (0, 1), xycoords="axes fraction", xytext=(0, 6), textcoords="offset points",
                    fontsize=10, ha="left", va="bottom")
    if letra:
        ax.annotate(letra, (0, 1), xycoords="axes fraction", xytext=(-46, 24), textcoords="offset points",
                    fontsize=11, fontweight="bold", ha="left", va="bottom")
    ax.set_xlabel(rotulo_x, fontsize=10)


def salvar(fig, nome: str):
    caminho = SAIDA / f"{nome}.png"
    fig.savefig(caminho, dpi=180, bbox_inches="tight", pad_inches=0.08)
    plt.close(fig)
    Image.open(caminho).convert("L").save(caminho, optimize=True)


def letras(n: int) -> list[str]:
    return [f"({chr(ord('a') + i)})" for i in range(n)]


def grade(n: int, colunas: int, largura: float = 7.5, altura_painel: float = 2.6):
    linhas = math.ceil(n / colunas)
    fig, eixos = plt.subplots(linhas, colunas, figsize=(largura, altura_painel * linhas), squeeze=False)
    for ax in eixos.flat[n:]:
        ax.set_visible(False)
    return fig, eixos.flat[:n]


def histogramas(df: pd.DataFrame, colunas: dict[str, str], nome: str, colunas_grade: int = 3):
    fig, eixos = grade(len(colunas), colunas_grade)
    for ax, (coluna, rotulo), letra in zip(eixos, colunas.items(), letras(len(colunas))):
        valores = df[coluna].to_numpy(dtype=float)
        if np.unique(valores).size <= 20:
            contagens = pd.Series(valores).value_counts().sort_index()
            ax.bar(contagens.index, contagens.values, width=0.8, color=PREENCHIMENTO, edgecolor=CINZA,
     linewidth=0.6)
        else:
            ax.hist(valores, bins=40, color=PREENCHIMENTO, edgecolor=CINZA, linewidth=0.5)
        assimetria = f"{stats.skew(valores):.2f}".replace(".", ",")
        ax.text(0.98, 0.95, f"assimetria = {assimetria}", transform=ax.transAxes,
                ha="right", va="top", fontsize=8.5, color=CINZA)
        painel(ax, rotulo, "Frequência", "", letra)
```

```
    fig.tight_layout(h_pad=3.2, w_pad=2.2)
    salvar(fig, nome)


def barras(df: pd.DataFrame, colunas: dict[str, str], nome: str):
    fig, eixos = grade(len(colunas), 2, altura_painel=2.5)
    for ax, (coluna, rotulo), letra in zip(eixos, colunas.items(), letras(len(colunas))):
        contagens = df[coluna].value_counts().sort_values()
        ax.barh(contagens.index.astype(str), contagens.values, color=PREENCHIMENTO, edgecolor=CINZA,
     linewidth=0.6)
        for i, v in enumerate(contagens.values):
            ax.text(v, i, f"  {v} ({100 * v / len(df):.1f}%)".replace(".", ","), va="center", fontsize=8.5,
      color=CINZA)
        ax.set_xlim(0, contagens.max() * 1.45)
        ax.grid(axis="x", color=GRADE)
        ax.grid(axis="y", visible=False)
        painel(ax, rotulo, "", "Número de imóveis", letra)
    fig.tight_layout(h_pad=3.0, w_pad=2.5)
    salvar(fig, nome)


def matriz_dispersao(df: pd.DataFrame, colunas: dict[str, str], nome: str, amostra: int = 3000, semente:
     int = 1):
    """Matriz adaptada: dispersões abaixo da diagonal, histogramas na diagonal e r de Pearson acima."""
    nomes = list(colunas)
    k = len(nomes)
    pontos = df.sample(min(amostra, len(df)), random_state=semente)
    correlacoes = df[nomes].corr()
    fig, eixos = plt.subplots(k, k, figsize=(7.6, 7.6))
    for i, linha in enumerate(nomes):
        for j, coluna in enumerate(nomes):
            ax = eixos[i, j]
            ax.grid(False)
            ax.set_xticks([])
            ax.set_yticks([])
            if i == j:
                ax.hist(df[linha], bins=25, color=PREENCHIMENTO, edgecolor=CINZA, linewidth=0.4)
                ax.spines["left"].set_visible(False)
            elif i > j:
                jitter = np.random.default_rng(semente).uniform(-0.15, 0.15, size=(len(pontos), 2))
                x = pontos[coluna] + (jitter[:, 0] if df[coluna].nunique() <= 20 else 0)
                y = pontos[linha] + (jitter[:, 1] if df[linha].nunique() <= 20 else 0)
                ax.scatter(x, y, s=1.2, color=PRETO, alpha=0.18, linewidths=0, rasterized=True)
            else:
                r = correlacoes.loc[linha, coluna]
                ax.text(0.5, 0.5, f"{r:.2f}".replace(".", ","), transform=ax.transAxes, ha="center", va="
     center",
                        fontsize=8 + 7 * abs(r), fontweight="bold" if abs(r) >= 0.5 else "normal",
                        color=PRETO if abs(r) >= 0.3 else CINZA)
                for lado in ("left", "bottom"):
                    ax.spines[lado].set_visible(False)
            if j == 0:
                ax.set_ylabel(colunas[linha], fontsize=8.5, rotation=0, ha="right", va="center", labelpad
     =4)
            if i == k - 1:
                ax.set_xlabel(colunas[coluna], fontsize=8.5, rotation=30, ha="right")
    fig.subplots_adjust(wspace=0.08, hspace=0.08, left=0.15, bottom=0.13, right=0.99, top=0.99)
    salvar(fig, nome)


def boxplots(df: pd.DataFrame, resposta: str, rotulo_resposta: str, categorias: dict[str, str], nome: str):
    fig, eixos = grade(len(categorias), 2, altura_painel=2.9)
    for ax, (coluna, rotulo), letra in zip(eixos, categorias.items(), letras(len(categorias))):
        niveis = df.groupby(coluna, observed=True)[resposta].median().sort_values().index
        dados = [df.loc[df[coluna] == n, resposta] for n in niveis]
        ax.boxplot(dados, vert=False, widths=0.6, patch_artist=True, showfliers=True,
                   boxprops=dict(facecolor=PREENCHIMENTO, edgecolor=CINZA),
                   medianprops=dict(color=PRETO, linewidth=1.6), whiskerprops=dict(color=CINZA),
                   capprops=dict(color=CINZA),
                   flierprops=dict(marker="o", markersize=1.8, markerfacecolor=PRETO, markeredgecolor="none
     ", alpha=0.35))
        ax.set_yticks(range(1, len(niveis) + 1), [str(n) for n in niveis])
        ax.grid(axis="x", color=GRADE)
        ax.grid(axis="y", visible=False)
```

```
        painel(ax, rotulo, "", rotulo_resposta, letra)
    fig.tight_layout(h_pad=3.0, w_pad=2.5)
    salvar(fig, nome)


def diagnostico(modelo, nome: str, maximo_pontos: int | None = None):
    """Histograma com densidade normal, resíduos versus preditos e Q-Q (três painéis)."""
    residuos = np.asarray(modelo.resid)
    preditos = np.asarray(modelo.fittedvalues)
    sigma = np.sqrt(modelo.scale)
    fig = plt.figure(figsize=(7.5, 6.4))
    ax1 = fig.add_axes([0.08, 0.57, 0.39, 0.33])
    ax2 = fig.add_axes([0.58, 0.57, 0.39, 0.33])
    ax3 = fig.add_axes([0.33, 0.07, 0.39, 0.33])

    contagens, bordas, _ = ax1.hist(residuos, bins=45, color=PREENCHIMENTO, edgecolor=CINZA, linewidth=0.4)
    xs = np.linspace(bordas[0], bordas[-1], 300)
    ax1.plot(xs, stats.norm.pdf(xs, 0, sigma) * len(residuos) * (bordas[1] - bordas[0]), color=PRETO, lw
    =1.8,
             label="Densidade normal")
    ax1.legend(loc="upper right", fontsize=8.5)
    painel(ax1, "Histograma e densidade normal", "Frequência", "Resíduos", "(a)")

    ax2.scatter(preditos, residuos, s=2, color=PRETO, alpha=0.25, linewidths=0, rasterized=True)
    ax2.axhline(0, color=REFERENCIA, lw=1.2, ls="--")
    suave = lowess(residuos, preditos, frac=0.2, it=0, return_sorted=True)
    ax2.plot(suave[:, 0], suave[:, 1], color=PRETO, lw=1.6, label="LOWESS")
    ax2.legend(loc="upper right", fontsize=8.5)
    painel(ax2, "Resíduos versus valores preditos", "Resíduos", "Valores preditos", "(b)")

    ordenados = np.sort(residuos)
    n = len(ordenados)
    teoricos = stats.norm.ppf((np.arange(1, n + 1) - 0.375) / (n + 0.25))
    q_x, q_y = np.quantile(teoricos, [0.25, 0.75]), np.quantile(ordenados, [0.25, 0.75])
    inclinacao = (q_y[1] - q_y[0]) / (q_x[1] - q_x[0])
    intercepto = q_y[0] - inclinacao * q_x[0]
    ax3.scatter(teoricos, ordenados, s=2.5, color=PRETO, linewidths=0, rasterized=True)
    limites = np.array([teoricos.min(), teoricos.max()])
    ax3.plot(limites, intercepto + inclinacao * limites, color=REFERENCIA, lw=1.1)
    painel(ax3, "Gráfico Q-Q dos resíduos", "Quantis dos resíduos", "Quantis teóricos N(0,1)", "(c)")
    salvar(fig, nome)


ABREVIACOES = {"São Paulo": "SP", "Rio de Janeiro": "RJ", "Belo Horizonte": "BH", "Porto Alegre": "POA",
               "Campinas": "CPS", "Não mobiliado": "Não mob.", "Apartamento": "Apto."}


def residuos_preditores(modelo, df: pd.DataFrame, numericas: dict[str, str], categoricas: dict[str, str],
     nome: str):
    """Resíduos contra cada preditora, com LOWESS para as numéricas e boxplots para as categóricas."""
    residuos = np.asarray(modelo.resid)
    total = len(numericas) + len(categoricas)
    fig, eixos = grade(total, 3, altura_painel=2.45)
    rotulos = letras(total)
    for ax, (coluna, rotulo), letra in zip(eixos, numericas.items(), rotulos):
        x = df[coluna].to_numpy(dtype=float)
        desloc = np.random.default_rng(3).uniform(-0.18, 0.18, len(x)) if np.unique(x).size <= 30 else 0
        ax.scatter(x + desloc, residuos, s=1.2, color=PRETO, alpha=0.15, linewidths=0, rasterized=True)
        ax.axhline(0, color=REFERENCIA, lw=1.0, ls="--")
        suave = lowess(residuos, x, frac=0.3, it=0, return_sorted=True)
        ax.plot(suave[:, 0], suave[:, 1], color=PRETO, lw=1.6)
        painel(ax, rotulo, "Resíduos", "", letra)
    for ax, (coluna, rotulo), letra in zip(eixos[len(numericas):], categoricas.items(), rotulos[len(
     numericas):]):
        niveis = list(df[coluna].cat.categories) if hasattr(df[coluna], "cat") else sorted(df[coluna].
     unique())
        dados = [residuos[(df[coluna] == n).to_numpy()] for n in niveis]
        ax.boxplot(dados, widths=0.6, patch_artist=True,
                   boxprops=dict(facecolor=PREENCHIMENTO, edgecolor=CINZA),
                   medianprops=dict(color=PRETO, linewidth=1.4), whiskerprops=dict(color=CINZA),
                   capprops=dict(color=CINZA),
                   flierprops=dict(marker="o", markersize=1.5, markerfacecolor=PRETO, markeredgecolor="none
     ", alpha=0.3))
```

```
        ax.set_xticks(range(1, len(niveis) + 1), [ABREVIACOES.get(str(n), str(n)) for n in niveis],
    fontsize=8.5)
        ax.axhline(0, color=REFERENCIA, lw=1.0, ls="--")
        painel(ax, rotulo, "Resíduos", "", letra)
    fig.tight_layout(h_pad=3.0, w_pad=1.8)
    salvar(fig, nome)


def influencia(modelo, nome: str):
    infl = modelo.get_influence()
    cook = infl.cooks_distance[0]
    alavancagem = infl.hat_matrix_diag
    estudentizados = infl.resid_studentized_external
    n, p = len(cook), int(modelo.df_model) + 1
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(7.5, 3.0))
    ax1.vlines(np.arange(n), 0, cook, color=PRETO, lw=0.5)
    ax1.axhline(4 / n, color=REFERENCIA, lw=1.0, ls="--", label="4/n")
    ax1.legend(loc="upper right", fontsize=8.5)
    painel(ax1, "Distância de Cook", "D de Cook", "Índice do registro", "(a)")
    ax2.scatter(alavancagem, estudentizados, s=2, color=PRETO, alpha=0.3, linewidths=0, rasterized=True)
    ax2.axvline(2 * p / n, color=REFERENCIA, lw=1.0, ls="--", label="2p/n")
    for limite in (-3, 3):
        ax2.axhline(limite, color=REFERENCIA, lw=1.0, ls=":")
    ax2.legend(loc="upper right", fontsize=8.5)
    painel(ax2, "Alavancagem e resíduos", "Resíduo estudentizado", "Alavancagem h", "(b)")
    fig.tight_layout(w_pad=3.0)
    salvar(fig, nome)


def comparacao_escalas(modelo_original, modelo_final, nome: str):
    fig, eixos = plt.subplots(1, 2, figsize=(7.5, 3.1))
    for ax, modelo, titulo, letra in zip(eixos, (modelo_original, modelo_final),
                                         ("Aluguel em R$ (sem transformações)", "log10(Aluguel) (modelo
    final)"),
                                         ("(a)", "(b)")):
        ax.scatter(modelo.fittedvalues, modelo.resid, s=2, color=PRETO, alpha=0.25, linewidths=0,
    rasterized=True)
        ax.axhline(0, color=REFERENCIA, lw=1.0, ls="--")
        painel(ax, titulo, "Resíduos", "Valores preditos", letra)
    eixos[0].ticklabel_format(style="plain")
    fig.tight_layout(w_pad=3.0)
    salvar(fig, nome)
```